
神经网络语言模型的 Scaling Laws

Jared Kaplan *

Johns Hopkins University, OpenAI
jaredk@jhu.edu

Sam McCandlish*

OpenAI
sam@openai.com

Tom Henighan

OpenAI
henighan@openai.com

Tom B. Brown

OpenAI
tom@openai.com

Benjamin Chess

OpenAI
bchess@openai.com

Rewon Child

OpenAI
rewon@openai.com

Scott Gray

OpenAI
scott@openai.com

Alec Radford

OpenAI
alec@openai.com

Jeffrey Wu

OpenAI
jeffwu@openai.com

Dario Amodei

OpenAI
damodei@openai.com

Abstract

我们研究了语言模型在交叉熵损失上的经验 Scaling Laws。损失随模型规模、数据集规模和训练所用计算量呈幂律关系缩放，部分趋势跨越了七个数量级以上。其他架构细节（如网络宽度或深度）在较宽范围内影响极小。简单的方程描述了过拟合对模型/数据集规模的依赖关系以及训练速度对模型规模的依赖关系。这些关系使我们能够确定固定计算预算下的最优分配方案。更大的模型在样本效率上显著更优，因此，以计算效率最优的方式训练意味着：在相对较少的数据上训练非常大的模型，并在收敛之前就显著提前停止。

*Equal contribution.

Contributions: Jared Kaplan and Sam McCandlish led the research. Tom Henighan contributed the LSTM experiments. Tom Brown, Rewon Child, and Scott Gray, and Alec Radford developed the optimized Transformer implementation. Jeff Wu, Benjamin Chess, and Alec Radford developed the text datasets. Dario Amodei provided guidance throughout the project.

Contents

1 引言	2
2 背景及方法	6
3 经验结果与基本幂律	7
4 描绘无限数据极限与过拟合	9
5 模型规模和训练时间的 Scaling Laws	11
6 计算预算的最优分配	13
7 相关工作	16
8 讨论	17
Appendices	18
A Summary of Power Laws	18
B Empirical Model of Compute-Efficient Frontier	18
C Caveats	20
D Supplemental Figures	21

1 引言

语言为人工智能的研究提供了天然的领域，因为绝大多数推理任务都可以用语言高效地表达和评估，而全球范围内的文本数据则通过生成式建模为无监督学习提供了丰富的数据资源。深度学习近年来在语言建模方面取得了快速进展，当前最先进的模型 [RNSS18, DCLT18, YDY⁺19, LOG⁺19, RSR⁺19] 在多项具体任务上已接近人类水平的表现 [WPN⁺19]，包括根据提示生成连贯的多段落文本 [RWC⁺19]。

人们可能会预期语言建模性能取决于模型架构、神经模型的规模、训练所使用的计算能力以及训练过程可用的数据。本文中，我们将实证研究语言建模损失对所有这些因素的依赖关系，重点关注 Transformer 架构 [VSP⁺17, LSP⁺18]。语言任务性能的高上限和低下限使我们能够在超过七个数量级的规模范围内研究趋势。

在整篇论文中，我们将观察到性能作为训练时间、上下文长度、数据集规模、模型规模和计算预算的函数的精确幂律 Scaling 关系。

1.1 总结

我们对 Transformer 语言模型的主要发现如下：

性能在很大程度上取决于规模，而对模型形状的依赖较弱： 模型性能最强烈地依赖于规模，规模由三个因素构成：模型参数数量 N （不包括嵌入参数）、数据集规模 D 和训练所用计算量 C 。在合理范围内，性能对深度 vs. 宽度等其他架构超参数的依赖非常弱。（第 3 节）

²此处我们展示的是使用足够小 batch size 时的预测计算量。与纯经验数据的对比请参见图 13。

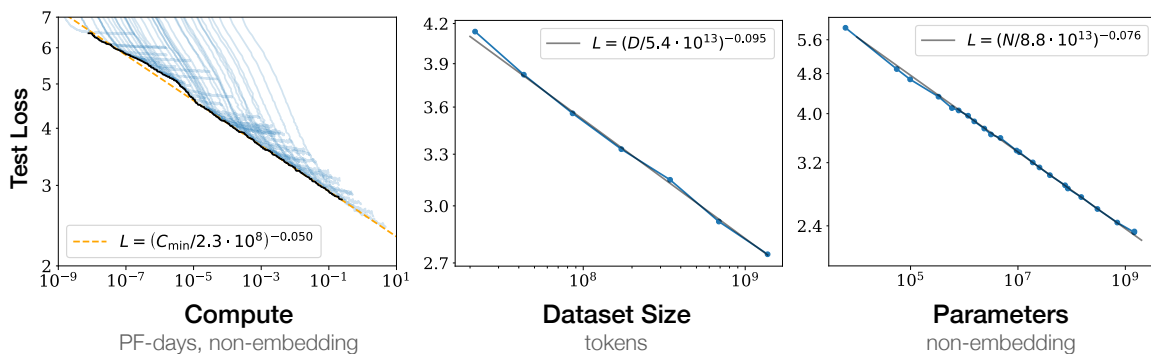


Figure 1 语言模型性能随着模型规模、数据集规模和训练所用计算量的增加而平滑改善² 用于训练。为了获得最优性能，所有三个因素必须协同扩展。当不受其他两个因素瓶颈限制时，经验性能与每个单因素呈幂律关系。

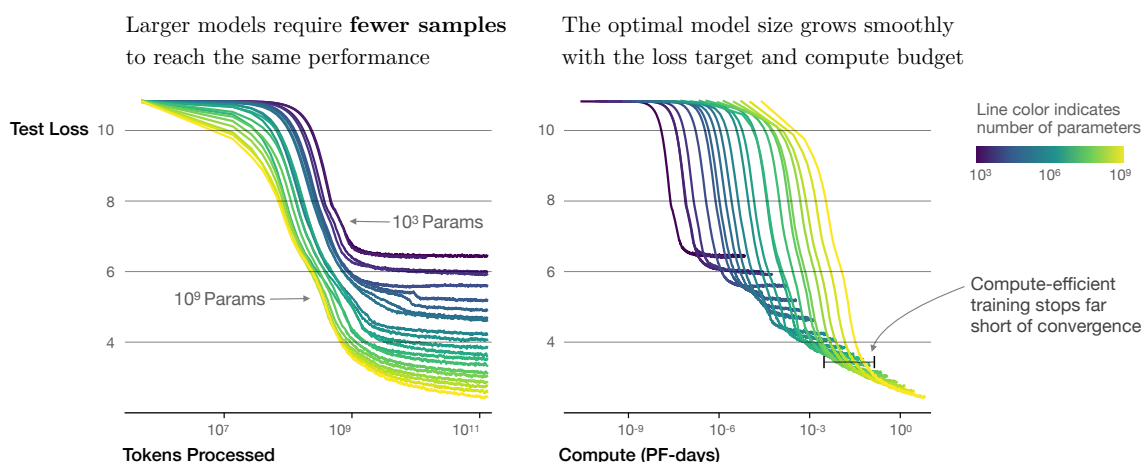


Figure 2 我们展示了一系列语言模型训练运行，模型规模从 10^3 到 10^9 个参数（不包括嵌入参数）。

平滑的幂律关系： 当不受其他两个因素瓶颈限制时，性能与每个规模因子 N, D, C 呈幂律关系，趋势跨越六个数量级以上（见图 1）。我们在上限端未观察到偏离这些趋势的迹象，尽管性能最终在达到零损失之前必然会趋于平缓。（第 3 节）

过拟合的普适性： 只要我们以协调的方式同时扩展 N 和 D ，性能就会可预测地改善；但如果固定其中一个而不断增加另一个，则会进入收益递减的区域。性能惩罚可预测地依赖于比值 $N^{0.74}/D$ ，这意味着每当我们把模型规模扩大 8 倍时，只需要将数据增加约 5 倍即可避免性能惩罚。（第 4 节）

训练的普适性： 训练曲线遵循可预测的幂律，其参数大致与模型规模无关。通过外推训练曲线的早期部分，我们可以大致预测如果训练更长时间将能达到的损失。（第 5 节）

迁移能力随测试性能提升： 当我们在与训练数据分布不同的文本上评估模型时，结果与训练验证集上的结果强相关，且损失存在大致恒定的偏移——换言之，向不同分布的迁移会产生恒定的惩罚，但除此之外，性能大致与训练集上的表现同步提升。（第 3.2.2 节）

样本效率： 大模型比小模型更具样本效率，能够以更少的优化步数（图 2）和更少的数据量（图 4）达到相同的性能水平。

收敛是低效的： 当在固定的计算预算 C 内工作，但对模型规模 N 或可用数据 D 没有其他限制时，我们通过训练非常大的模型并在显著早于收敛时停止训练来获得最优性能（见图 3）。因此，最大化计算效率的训练将比基于训练小模型至收敛所预期的更加样本高效，数据需求随训练计算量增长非常缓慢，满足 $D \sim C^{0.27}$ 。（第 6 节）

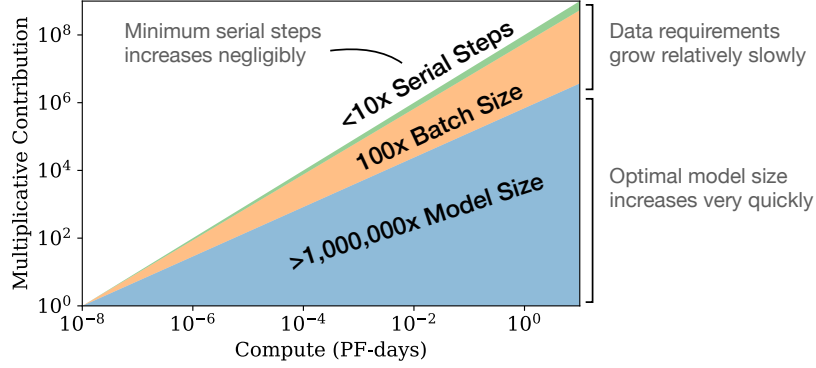


Figure 3 随着可用计算资源的增加，我们可以选择如何将其分配用于训练更大的模型、使用更大的 batch size 以及训练更多的步数。我们以计算量增加十亿倍为例对此进行了说明。对于计算效率最优的训练，大部分增加的计算量应投入到模型规模的扩大上。为避免数据重复使用，仅需相对较小的数据增量。在增加的数据中，大部分可以通过更大的 batch size 来提高并行度，只需非常小的串行训练时间增量。

最优 batch size: 训练这些模型的理想 batch size 大致仅与损失呈幂律关系，并且可以通过测量梯度噪声尺度 [MKAT18] 持续确定；对于我们能够训练的最大模型，在收敛时其值大约为 100 万到 200 万个 tokens。（第 5.1 节）

综合来看，这些结果表明，当我们在模型规模、数据和计算之间进行适当扩展时，语言建模性能会平滑且可预测地改善。我们预期更大的语言模型将表现得更好，并且比当前模型更具样本效率。

1.2 Scaling Laws 总结

对语言进行自回归建模的 Transformer 的测试损失，在性能仅受非嵌入参数数量 N 、数据集规模 D 或最优分配的计算预算 $C_{\min}$ 限制时，可以通过幂律预测（见图 1）：

1. 对于参数数量有限的模型，在足够大的数据集上训练至收敛：

$$L(N) = (N_c/N)^{\alpha_N} ; \alpha_N \sim 0.076, N_c \sim 8.8 \times 10^{13} \text{ (non-embedding parameters)} \quad (1.1)$$

2. 对于使用有限数据集训练的大模型，配合 early stopping：

$$L(D) = (D_c/D)^{\alpha_D} ; \alpha_D \sim 0.095, D_c \sim 5.4 \times 10^{13} \text{ (tokens)} \quad (1.2)$$

3. 当在有限计算量、足够大的数据集、最优规模模型和足够小的 batch size 条件下训练时（实现计算的最优利用³）：

$$L(C_{\min}) = (C_c^{\min}/C_{\min})^{\alpha_C^{\min}} ; \alpha_C^{\min} \sim 0.050, C_c^{\min} \sim 3.1 \times 10^8 \text{ (PF-days)} \quad (1.3)$$

这些关系在 $C_{\min}$ 的八个数量级、 N 的六个数量级以及 D 的两个以上数量级范围内成立。它们对模型形状和其他 Transformer 超参数（深度、宽度、自注意力头数）的依赖非常弱，具体数值与 Webtext2 训练集 [RWC⁺19] 相关联。幂律指数 $\alpha_N, \alpha_D, \alpha_C^{\min}$ 指定了当我们扩展 N 、 D 或 $C_{\min}$ 时预期的性能改善程度；例如，将参数数量翻倍可使损失缩小 $2^{-\alpha_N} = 0.95$ 倍。 $N_c, C_c^{\min}, D_c$ 的精确数值取决于词表规模和分词方式，因此不具有根本性含义。

临界 batch size 决定了数据并行的速度/效率权衡 ([MKAT18])，它也大致服从关于 L 的幂律关系：

$$B_{\text{crit}}(L) = \frac{B_*}{L^{1/\alpha_B}}, \quad B_* \sim 2 \cdot 10^8 \text{ tokens}, \alpha_B \sim 0.21 \quad (1.4)$$

方程 (1.1) 和 (1.2) 共同表明，随着模型规模的增加，我们应按 $D \propto N^{\frac{\alpha_N}{\alpha_D}} \sim N^{0.74}$ 亚线性地增加数据集规模。实际上，我们发现存在一个结合了 (1.1) 和 (1.2) 的单一方程，它同时支配了对 N 和 D 的依赖关系以及过拟合的程度：

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \quad (1.5)$$

³我们也在固定 batch size 训练时观察到训练计算量 C 的经验幂律趋势（图 1），但应用于预测的应是 $C_{\min}$ 的趋势。它们通过方程 (5.5) 关联。

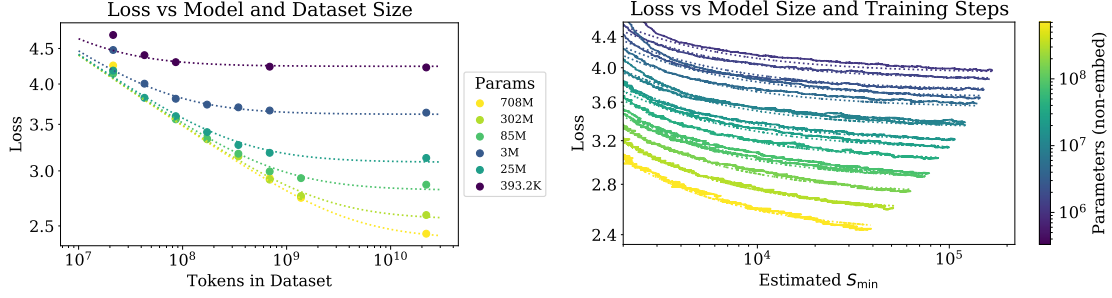


Figure 4 左: early-stopped 的测试损失 $L(N, D)$ 根据方程 (1.5) 可预测地随数据集规模 D 和模型规模 N 变化。右: 在初始过渡期之后, 所有模型规模 N 的学习曲线都可以用方程 (1.6) 拟合, 该方程以 $S_{\min}$ 参数化, $S_{\min}$ 是在大 batch size 下训练的步数 (详见第 5.1 节)。

其拟合结果如图 4 左图所示。我们推测这一函数形式也可能参数化其他生成建模任务的训练对数似然。当在无限数据限制下对一个给定模型进行有限次参数更新步数 S 的训练时, 在初始过渡期之后, 学习曲线可以准确地由以下方程拟合 (见图 4 右图)

$$L(N, S) = \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}(S)} \right)^{\alpha_S} \quad (1.6)$$

其中 $S_c \approx 2.1 \times 10^3$, $\alpha_S \approx 0.76$, $S_{\min}(S)$ 是使用方程 (5.4) 估计的最小可能优化步数 (参数更新数)。

当在固定计算预算 C 内训练且无其他约束时, 方程 (1.6) 给出了如下预测: 最优模型规模 N 、最优 batch size B 、最优步数 S 和数据集规模 D 应满足

$$N \propto C^{\alpha_C^{\min}/\alpha_N}, \quad B \propto C^{\alpha_C^{\min}/\alpha_B}, \quad S \propto C^{\alpha_C^{\min}/\alpha_S}, \quad D = B \cdot S \quad (1.7)$$

with

$$\alpha_C^{\min} = 1/(1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \quad (1.8)$$

which closely matches the empirically optimal results $N \propto C_{\min}^{0.73}$ 、 $B \propto C_{\min}^{0.24}$ 和 $S \propto C_{\min}^{0.03}$ 高度吻合。随着计算预算 C 的增加, 应将其主要用于扩大模型规模, 而非大幅增加训练时间或数据集规模 (见图 3)。这还意味着随着模型规模的增长, 其样本效率不断提高。在实践中, 由于硬件约束, 研究者通常训练较小模型的时间比最大化计算效率所需的时间更长。最优性能以幂律形式依赖于总计算量 (见方程 (1.3))。

对方程 (1.5) 提供了一些基本的理论动机, 对学习曲线拟合及其对训练时间的影响进行了分析, 并给出了按 token 分解的结果。我们还对 LSTM 和循环 Transformer [DGV⁺18] 进行了简要比较。

1.3 符号定义

我们使用以下符号:

- L – 交叉熵损失 (以 nats 为单位)。通常它在上下文中的 tokens 上进行平均, 但在某些情况下我们报告上下文中特定 tokens 的损失。
- N – 模型参数的数量, **不包括所有表 and 位置嵌入**
- $C \approx 6NBS$ – 总非嵌入训练计算量的估计, 其中 B 是 batch size, S 是训练步数 (即参数更新数)。我们以 PF-days 为单位给出数值, 其中 $1 \text{ PF-day} = 10^{15} \times 24 \times 3600 = 8.64 \times 10^{19}$ 次浮点运算。
- D – 以 tokens 计的数据集规模
- B_{crit} – 临界 batch size [MKAT18], 在第 5.1 节定义和讨论。以临界 batch size 训练可在时间和计算效率之间取得大致最优的折衷。
- $C_{\min}$ – 达到给定损失值所需的最小非嵌入计算量的估计。这是当模型以远小于临界 batch size 的 batch size 训练时所使用的训练计算量。
- $S_{\min}$ – 达到给定损失值所需的最小训练步数的估计。这也是当模型以远大于临界 batch size 的 batch size 训练时所使用的训练步数。
- α_X – 损失缩放 $L(X) \propto 1/X^{\alpha_X}$ 的幂律指数, 其中 X 可以是 $N, D, C, S, B, C^{\min}$ 中的任意一个。

操作	参数	每个 Token 的 FLOPs
Embed	$(n_{\text{vocab}} + n_{\text{ctx}}) d_{\text{model}}$	$4d_{\text{model}}$
Attention: QKV	$n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$	$2n_{\text{layer}} d_{\text{model}} 3d_{\text{attn}}$
Attention: Mask	—	$2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$
Attention: Project	$n_{\text{layer}} d_{\text{attn}} d_{\text{model}}$	$2n_{\text{layer}} d_{\text{attn}} d_{\text{embd}}$
Feedforward	$n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$	$2n_{\text{layer}} 2d_{\text{model}} d_{\text{ff}}$
De-embed	—	$2d_{\text{model}} n_{\text{vocab}}$
Total (Non-Embedding)	$N = 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}})$	$C_{\text{forward}} = 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$

Table 1 Transformer 模型的参数数量和计算量（前向传递）估计。省略了非线性、偏置和层归一化等次主导项。

2 背景及方法

我们在 WebText2 上训练语言模型，WebText2 是 WebText [RWC⁺19] 数据集的扩展版本，使用 byte-pair encoding [SHB15] 进行分词，词表规模为 $n_{\text{vocab}} = 50257$ 。我们优化在 1024 token 上下文上平均的自回归对数似然（即交叉熵损失），这也是我们的主要性能指标。我们记录 WebText2 测试分布以及若干其他文本分布上的损失。我们主要训练 decoder-only [LSP⁺18, RNSS18] Transformer [VSP⁺17] 模型，但也训练了 LSTM 模型和 Universal Transformer [DGV⁺18] 以进行比较。

2.1 Transformer 的参数与计算 Scaling

我们使用超参数 n_{layer} (层数)、 d_{model} (残差流维度)、 d_{ff} (中间前馈层维度)、 d_{attn} (注意力输出维度) 和 n_{heads} (每层注意力头数) 来参数化 Transformer 架构。我们在输入上下文中包含 n_{ctx} 个 token，除特别说明外 $n_{\text{ctx}} = 1024$ 。

我们用 N 表示模型规模，定义为其**非嵌入**参数的数量：

$$\begin{aligned}
N &\approx 2d_{\text{model}} n_{\text{layer}} (2d_{\text{attn}} + d_{\text{ff}}) \\
&= 12n_{\text{layer}} d_{\text{model}}^2 \quad \text{with the standard} \quad d_{\text{attn}} = d_{\text{ff}}/4 = d_{\text{model}}
\end{aligned} \tag{2.1}$$

其中我们排除了偏置和其他次主导项。我们的模型在嵌入矩阵中还有 $n_{\text{vocab}} d_{\text{model}}$ 个参数，并使用 $n_{\text{ctx}} d_{\text{model}}$ 个参数作为位置嵌入，但在讨论模型规模 N 时我们不包含这些参数；我们将看到这样可以产生更清晰的 Scaling Laws。

评估 Transformer 的一次前向传递大约需要

$$C_{\text{forward}} \approx 2N + 2n_{\text{layer}} n_{\text{ctx}} d_{\text{model}} \tag{2.2}$$

次加法-乘法运算，其中因子 2 来自矩阵乘法中使用的乘累加运算。更详细的逐操作参数和计算量统计见表 1。

对于满足 $d_{\text{model}} > n_{\text{ctx}}/12$ 的上下文和模型，每个 token 的上下文相关计算成本在总计算量中占比较小。由于我们主要研究 $d_{\text{model}} \gg n_{\text{ctx}}/12$ 的模型，我们在训练计算量估计中不包含上下文相关项。考虑到反向传递（计算量约为前向传递的两倍），我们将估计的非嵌入计算量定义为每个训练 token $C \approx 6N$ 次浮点运算。

2.2 训练过程

除非另有说明，我们使用 Adam 优化器 [KB14] 训练模型，固定步数为 2.5×10^5 ，batch size 为 512 个序列（每个序列 1024 个 token）。由于内存限制，我们最大的模型（超过 10 亿参数）使用 Adafactor [SS18] 进行训练。我们尝试了多种学习率和学习率调度方案，讨论见附录 D.6。我们发现收敛时的结果在很大程度上与学习率调度方案无关。除非另有说明，我们数据中包含的所有训练运行均采用 3000 步线性 warmup 随后余弦衰减到零的学习率调度方案。

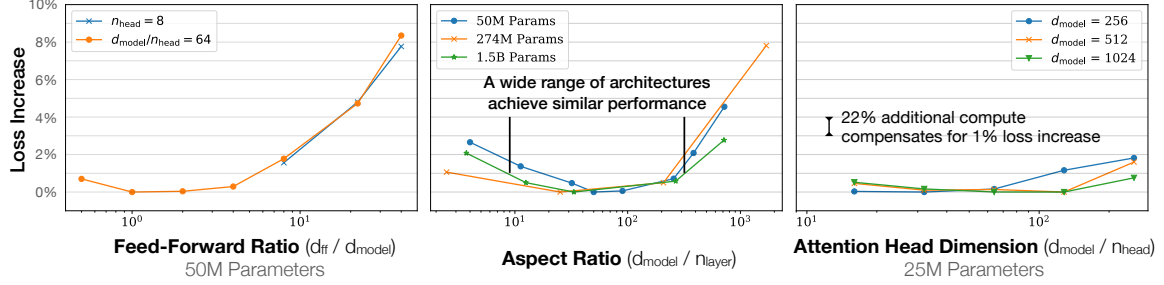


Figure 5 当总非嵌入参数数量 N 固定时，性能对模型形状的依赖非常轻微。在较宽的形状范围内，损失仅变化几个百分点。参数数量的细微差异通过对 $L(N)$ 的拟合作为基线来补偿。特别地，宽高比可以变化 40 倍而仅轻微影响性能；一个 $(n_{\text{layer}}, d_{\text{model}}) = (6, 4288)$ 的模型在损失上与 [RWC⁺19] 中使用的 $(48, 1600)$ 模型相差在 3% 以内。

2.3 数据集

我们在 WebText 数据集 [RWC⁺19] 的扩展版本上训练模型。原始 WebText 数据集是对 Reddit 上截至 2017 年 12 月、获得至少 3 karma 的外链进行网页爬取得到的。在第二个版本 WebText2 中，我们增加了 2018 年 1 月至 10 月期间的外链，同样要求至少 3 karma。karma 阈值作为人们是否认为该链接有趣或有用的启发式标准。新增链接的文本使用 Newspaper3k Python 库提取。数据集总共包含 2030 万篇文档，含有 96 GB 文本和 1.62×10^{10} 个单词（由 wc 定义）。然后我们应用 [RWC⁺19] 中描述的可逆分词器，产生 2.29×10^{10} 个 token。我们保留其中 6.6×10^8 个 token 作为测试集，同时还在类似准备的 Books Corpus [ZKZ⁺15]、Common Crawl [Fou]、英文 Wikipedia 以及一组公开可用的互联网图书样本上进行测试。

3 经验结果与基本幂律

为了刻画语言模型 Scaling 的特征，我们训练了多种多样的模型，调整了诸多因素，包括：

- 模型规模（范围从 768 到 15 亿个非嵌入参数）
- 数据集规模（范围从 2200 万到 230 亿个 tokens）
- 形状（包括深度、宽度、注意力头数和前馈层维度）
- 上下文长度（大多数运行行为 1024，但也用更短的上下文进行了实验）
- Batch size（大多数运行行为 2^{19} ，但也改变它以测量临界 batch size）

在本节中，我们将展示数据以及基于经验驱动的拟合，将理论分析留待后续章节进行。

3.1 近似的 Transformer 形状与超参数无关性

当总非嵌入参数数量 N 固定时，Transformer 性能对形状参数 $n_{\text{layer}}, n_{\text{heads}}$ 和 d_{ff} 的依赖非常弱。为验证这些结果，我们训练了固定规模的模型，每次仅改变一个超参数。对 n_{heads} 而言这最简单。当改变 n_{layer} 时，我们同时改变 d_{model} 以保持 $N \approx 12n_{\text{layer}}d_{\text{model}}^2$ 固定。类似地，要在固定模型规模下改变 d_{ff} ，我们也同时改变 d_{model} 参数，如表 1 中参数统计所要求。 n_{layers} 的独立性可以从更深层 Transformer 实际上表现为浅层模型的集成来理解，正如 ResNet [VWB16] 中所讨论的那样。结果如图 5 所示。

3.2 非嵌入参数数量 N 与性能的关系

在图 6 中，我们展示了各种模型的性能，从形状为 $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$ 的小模型到十亿参数级模型，形状范围从 $(6, 4288)$ 到 $(207, 768)$ 。这里我们在完整的 WebText2 数据集上训练至接近收敛，未观察到过拟合（除最大模型可能例外）。

如图 1 所示，我们发现随非嵌入参数数量 N 存在稳定的趋势，可以拟合为方程 (1.5) 的第一项，即

$$L(N) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} \quad (3.1)$$

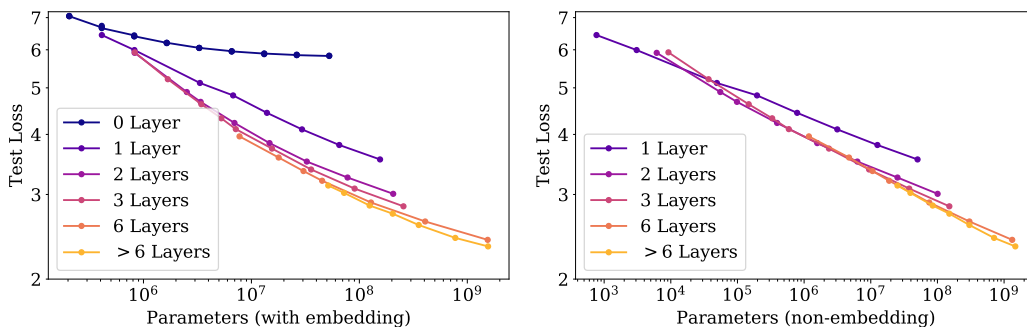


Figure 6 左: 当包含嵌入参数时, 性能除参数数量外似乎还强烈依赖于层数。右: 当排除嵌入参数时, 不同深度模型的性能收敛到同一趋势上。仅层数少于 2 层或深宽比极端的模型会显著偏离该趋势。

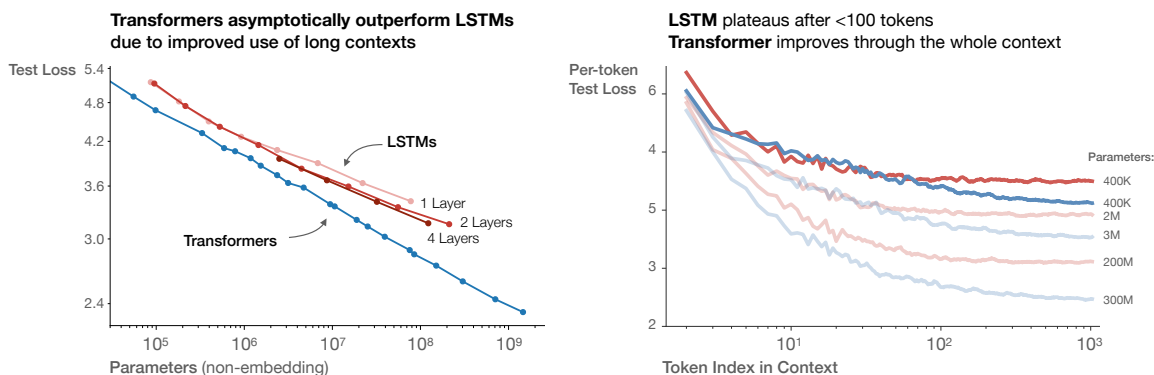


Figure 7

要观察这些趋势, 关键是将性能作为 N 的函数来研究; 如果我们改为使用总参数数量 (包含嵌入参数), 趋势会变得模糊 (见图 6)。这表明嵌入矩阵可以在不影响性能的情况下缩小, 正如最近的工作 [LCG⁺19] 所见。

尽管这些模型是在 WebText2 数据集上训练的, 但它们在各种其他数据集上的测试损失同样是 N 的幂律函数, 且指数几乎相同, 如图 8 所示。

3.2.1 与 LSTM 和 Universal Transformers 的比较

在图 7 中, 我们比较了 LSTM 和 Transformer 作为非嵌入参数数量 N 的函数的性能。LSTM 使用相同的数据集和上下文长度进行训练。我们从这些图中看到, LSTM 在上下文早期 token 上的表现与 Transformer 相当, 但在后续 token 上无法匹敌 Transformer 的性能。我们在附录 D.5 中给出了性能与上下文位置之间的幂律关系, 更大模型的幂指数越来越大, 表明其快速识别模式的能力在提高。

我们还在附录图 17 中比较了标准 Transformer 与循环 Transformer [DGV⁺18] 的性能。这些模型复用参数, 因此作为 N 的函数表现略好, 但代价是每个参数的计算量增加。

3.2.2 不同数据分布间的泛化

我们还在其他一些文本数据分布上测试了我们的模型。这些数据集上作为模型规模函数的测试损失如图 8 所示; 在所有情况下, 模型仅在 WebText2 数据集上训练。我们看到, 其他数据分布上的损失随模型规模平滑改善, 与 WebText2 上的改善直接平行。我们发现泛化性能几乎完全取决于分布内验证损失, 而不取决于训练时长或接近收敛的程度。我们同样未观察到对模型深度的依赖 (见附录 D.8)。

3.3 数据集规模和计算量对性能的影响

我们在图 1 中展示了测试损失作为数据集规模 D (以 token 计) 和训练计算量 C 的函数的经验趋势。

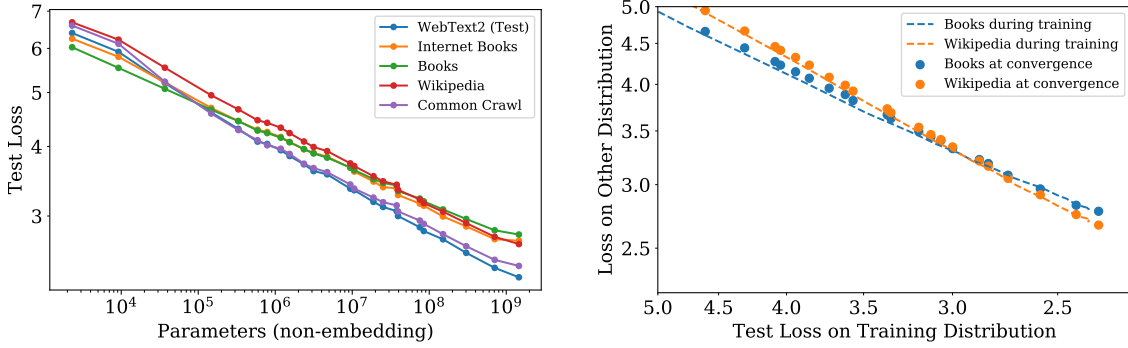


Figure 8 左: 对其他数据分布的泛化性能随模型规模平滑改善, 与 WebText2 训练分布之间仅存在一个很小且增长非常缓慢的偏移。右: 泛化性能仅取决于训练分布上的性能, 与训练所处阶段无关。我们比较了已收敛模型 (数据点) 与单个大模型在训练过程中 (虚线曲线) 的泛化能力。

对于 D 的趋势, 我们使用形状为 $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$ 的模型在 WebText2 数据集的固定子集上训练。我们在测试损失停止下降时停止训练。我们发现得到的测试损失可以用简单的幂律拟合

$$L(D) \approx \left(\frac{D_c}{D} \right)^{\alpha_D} \quad (3.2)$$

用于描述数据集规模的缩放。数据和拟合见 图 1。

训练期间使用的总非嵌入计算量可以估计为 $C = 6NBS$, 其中 B 是 batch size, S 是参数更新次数, 因子 6 考虑了前向和反向传递。因此对于给定的 C 值, 我们可以扫描所有具有不同 N 的模型, 以找到在步数 $S = \frac{C}{6BS}$ 时性能最佳的模型。注意在这些结果中, **所有模型的 batch size B 保持固定**, 这意味着这些经验结果并非真正最优。我们将在后续章节中使用调整后的 $C_{\min}$ 来产生更清晰的趋势。

结果以粗黑线出现在图 1 的左图中。它可以用以下公式拟合

$$L(C) \approx \left(\frac{C_c}{C} \right)^{\alpha_C} \quad (3.3)$$

该图还包括各个学习曲线的图像, 以阐明什么时候各个模型是最优的。我们稍后将更仔细地研究计算量的最优分配。数据强烈表明样本效率随模型规模提高, 我们也在附录图 19 中直接展示了这一点。

4 描绘无限数据极限与过拟合

在第 3 节中, 我们发现了语言建模性能的若干基本 Scaling Laws。在此我们将研究在同时改变 N 和 D 时, 规模为 N 的模型在用 D 个 token 训练后的性能。我们将经验性地证明, 最优训练的测试损失遵循方程 (1.5) 的 Scaling Law。这为在不断增加模型规模时控制过拟合所需的数据量提供了指导。

4.1 提出的 $L(N, D)$ 方程

我们根据三个原则选择了参数化形式 (1.5) (在此处重复以便于阅读):

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D} \quad (4.1)$$

1. 词表规模或 tokenization 的变化预计会对损失进行整体因子的缩放。 $L(N, D)$ 的参数化 (以及所有损失模型) 必须自然地允许这种缩放。
2. 固定 D 并令 $N \rightarrow \infty$, 总损失应趋近 $L(D)$ 。反之, 固定 N 并令 $D \rightarrow \infty$, 损失必须趋近 $L(N)$ 。
3. $L(N, D)$ 在 $D = \infty$ 处应当是解析的, 因此它具有关于 $1/D$ 的整数次幂的级数展开。对这一原则的理论支持明显弱于前两个原则。

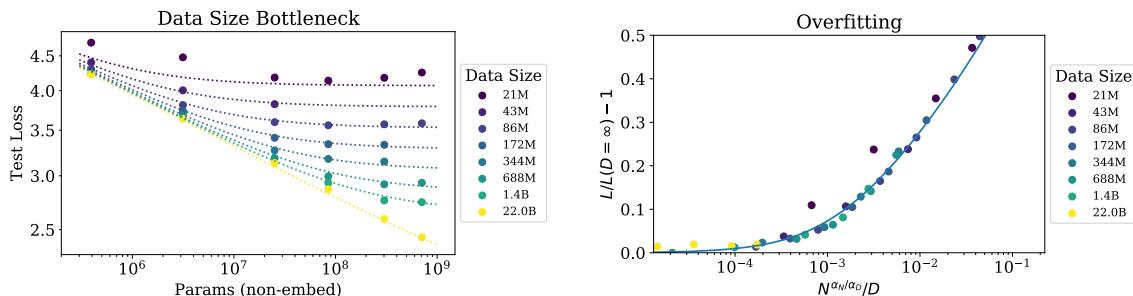


Figure 9 early-stopped 的测试损失 $L(N, D)$ 根据方程 (1.5) 可预测地依赖于数据集规模 D 和模型规模 N 。左: 在大的 D 下, 性能是 N 的直接幂律。对于较小的固定 D , 性能随 N 增加而停止改善, 模型开始过拟合。(反过来也成立, 见图 4。) 右: 过拟合的程度主要取决于比值 $N^{\frac{\alpha_N}{\alpha_D}}/D$, 正如方程 (4.3) 所预测。图中线条是我们对该方程的拟合。

我们对 $L(N, D)$ 的选择满足第一个要求, 因为我们可以通过改变词表来重新缩放 N_c, D_c 。这也意味着 N_c, D_c 的值没有根本性的含义。

由于我们在测试损失停止改善时进行 early stopping, 并以相同方式优化所有模型, 我们期望更大的模型应该总是表现得比更小的模型更好。但在固定的有限 D 下, 我们也不期望任何模型能够接近可能的最佳损失 (即文本的熵)。类似地, 固定规模的模型将受到容量限制。这些考虑激励了我们的第二个原则。注意, 知道无限 D 下的 $L(N)$ 和无限 N 下的 $L(D)$ 完全确定了 $L(N, D)$ 中的所有参数。

第三个原则更具推测性。有一个简单而普遍的理由可以预期过拟合在非常大的 D 下按 $\propto 1/D$ 缩放。过拟合应与数据集的方差或信噪比相关 [AS17], 而这按 $1/D$ 缩放。这一预期应对任何平滑的损失函数成立, 因为我们期望能够围绕 $D \rightarrow \infty$ 极限展开损失函数。然而, 这一论证假设 $1/D$ 校正项主导了其他方差来源, 如有限 batch size 和其他优化效能的限制。如果没有经验证实, 我们对其适用性不会非常有信心。

第三个原则解释了方程 (1.5) 中 N 和 D 角色之间的不对称性。非常类似的对称表达式⁴也有可能, 但它们不具备整数次幂的 $1/D$ 展开, 且需要引入额外的参数。

无论如何, 我们将看到我们的 $L(N, D)$ 方程能很好地拟合数据, 这也是我们的 $L(N, D)$ 假设最重要的理由。

4.2 结果

我们使用 10% 的 dropout 对所有模型进行正则化, 并通过跟踪测试损失并在其不再下降时停止训练。结果显示在图 9 中, 包括对方程 (1.5) 中四个参数 $\alpha_N, \alpha_D, N_c, D_c$ 的拟合:

参数	α_N	α_D	N_c	D_c
值	0.076	0.103	6.4×10^{13}	1.8×10^{13}

Table 2 $L(N, D)$ 的拟合结果

我们获得了极好的拟合效果, 唯一的例外是数据集被缩减了 1024 倍至约 2×10^7 个 token 的运行。对于如此小的数据集, 一个 epoch 仅包含 40 次参数更新。也许如此小的数据集代表了语言建模的另一种机制, 因为过拟合在训练非常早期就发生了 (见图 16)。另请注意, 这些参数与第 3 节中得到的参数有微小的差异, 因为这里我们拟合的是完整的 $L(N, D)$, 而不仅仅是 $L(N, \infty)$ 或 $L(\infty, D)$ 。

为了描绘无限数据极限的边界, 我们可以直接研究过拟合的程度。除了最大的模型外, 我们在完整的 220 亿 token WebText2 数据集上训练时没有看到过拟合的迹象, 因此我们可以将其视为 $D = \infty$ 的代表。因此我们可以通过定义

$$\delta L(N, D) \equiv \frac{L(N, D)}{L(N, \infty)} - 1 \quad (4.2)$$

⁴例如, 可以使用 $L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{D_c}{D} \right)^{\alpha_D} \right]^\beta$, 但这样就不具备 $1/D$ 展开。

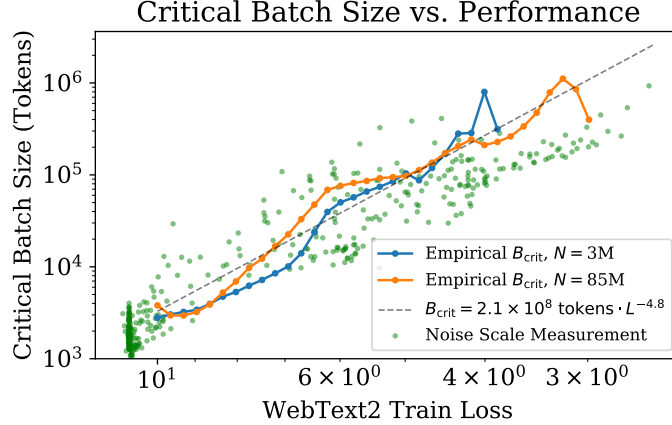


Figure 10 临界 batch size B_{crit} 随性能提升而遵循关于损失的幂律，且不直接依赖于模型规模。我们发现损失每下降 13%，临界 batch size 大约翻倍。 B_{crit} 是根据图 18 中的数据进行经验性测量的，但它也可以通过梯度噪声尺度大致预测，如 [MKAT18] 所述。

并研究其作为 N, D 的函数。实际上，我们经验性地看到 δL 仅依赖于 N 和 D 的某种特定组合，如图 16 所示。这源于方程 (1.5) 的 Scaling Law，该式暗含

$$\delta L \approx \left(1 + \left(\frac{N}{N_c} \right)^{\frac{\alpha_N}{\alpha_D}} \frac{D_c}{D} \right)^{\alpha_D} - 1 \quad (4.3)$$

注意在大的 D 下，该公式也有一组 $1/D$ 的幂级数展开。

我们估计不同随机种子下损失的变化约为 0.02，这意味着为了避免过拟合到收敛的该阈值范围内，我们需要

$$D \gtrsim (5 \times 10^3) N^{0.74} \quad (4.4)$$

根据这个关系，小于 10^9 个参数的模型可以在 220 亿 token 的 WebText2 数据集上以最小的过拟合进行训练，但我们最大的模型将遇到一些轻微过拟合。更一般地说，这个关系表明数据集规模可以亚线性地增长于模型规模，同时避免过拟合。然而，请注意这通常并不代表最大化计算效率的训练。我们还应当强调，我们没有在改变数据集和模型规模时优化正则化（例如 dropout 概率）。

5 模型规模和训练时间的 Scaling Laws

在本节中，我们将证明一个简单的 Scaling Law 可以很好地描述损失作为模型规模 N 和训练时间的函数。首先，我们将解释如何使用 [MKAT18] 的结果来定义一个通用的训练步数 S_{min} ，该定义考虑了我们大多数模型并非在最优 batch size 下训练的事实。然后，我们将证明可以使用方程 (1.6) 来拟合损失对模型规模和训练时间的依赖关系。之后，我们将使用这些结果来预测训练计算量在模型规模和训练时间之间的最优分配，并验证该预测。

5.1 对在 $B_{\text{crit}}(L)$ 下训练的调整

[MKAT18] (另见 [SLA⁺18, ZLN⁺19]) 提出了一个关于训练对 batch size 依赖关系的简洁经验理论。该工作论证了存在一个训练的临界 batch size B_{crit} ；对于 B 不超过 B_{crit} 的情况，增大 batch size 对计算效率的降低非常微小，而对于 $B > B_{\text{crit}}$ ，增大 B 会导致收益递减。该工作还论证了梯度噪声尺度为 B_{crit} 提供了一个简单的预测，且两者都不直接依赖于模型规模，仅通过已取得的损失值间接依赖。这些结果可用于预测训练时间和计算量如何随 batch size 变化。为了尽可能高效地利用训练时间和计算量，最好以 $B \approx B_{\text{crit}}$ 训练。以 $B \gg B_{\text{crit}}$ 训练可最小化训练步数，而以 $B \ll B_{\text{crit}}$ 训练可最小化计算量的使用。

更具体地说，该工作证明了对各种神经网络任务，训练步数 S 和处理的数据样本数 $E = BS$ 满足简单关系

$$\left(\frac{S}{S_{\text{min}}} - 1 \right) \left(\frac{E}{E_{\text{min}}} - 1 \right) = 1 \quad (5.1)$$

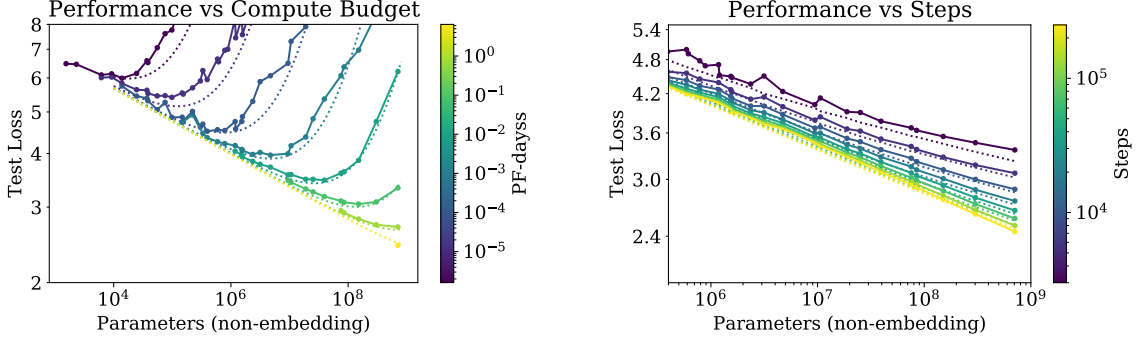


Figure 11 当我们固定总计算量或训练步数时，性能遵循方程 (5.6) 中的 $L(N, S)$ 。每个计算预算值都有一个能最大化性能的关联最优模型规模。在小的 S 值时拟合效果不理想并不奇怪，因为学习曲线的幂律方程在训练的非常早期会失效。

在训练至任意固定损失值 L 时成立。这里 $S_{\min}$ 是达到 L 所需的最小步数，而 $E_{\min}$ 是必须处理的最小数据样本数。

我们在附录图 18 中展示了 Transformer 的关系式 (5.1)。该关系定义了临界 batch size

$$B_{\text{crit}}(L) \equiv \frac{E_{\min}}{S_{\min}} \quad (5.2)$$

它是目标损失值的函数。以临界 batch size 训练可在时间/计算之间取得大致最优的折衷，需要 $2S_{\min}$ 个训练步数并处理 $E = 2E_{\min}$ 个数据样本。

在图 10 中，我们绘制了两个不同模型的临界 batch size 和梯度噪声尺度⁵ 作为训练损失的函数。我们看到 $B_{\text{crit}}(L)$ 与模型规模无关，仅依赖于损失 L 。因此 [MKAT18] 的预测对 Transformer 语言模型仍然成立。临界 batch size 可以拟合为关于损失的幂律

$$B_{\text{crit}}(L) \approx \frac{B_*}{L^{1/\alpha_B}} \quad (5.3)$$

其中 $B_* \approx 2 \times 10^8$ ， $\alpha_B \approx 0.21$ 。

我们选择这种 $B_{\text{crit}}(L)$ 参数化形式，是因为当损失趋近其最小值 $L_{\min}$ 时，梯度噪声尺度预期会发散，而我们预期 B_{crit} 会跟踪该噪声尺度。我们不知道 $L_{\min}$ 的值，因为我们没有看到模型趋近于它的迹象，但 $L_{\min} > 0$ ，因为自然语言的熵非零。由于 $L_{\min}$ 显然远小于我们已取得的 L 值，我们使用了当 $L \rightarrow 0$ 时 B_{crit} 发散的参数化形式。

我们将使用 $B_{\text{crit}}(L)$ 来估计在以 batch size $B = 2^{19}$ 个 token 训练时的训练步数 S 与以 $B \gg B_{\text{crit}}$ 训练时的训练步数之间的关系。这很简单

$$S_{\min}(S) \equiv \frac{S}{1 + B_{\text{crit}}(L)/B} \quad (\text{minimum steps, at } B \gg B_{\text{crit}}) \quad (5.4)$$

对于任意给定的目标损失值 L 成立。这也定义了如果我们以 $B \ll B_{\text{crit}}(L)$ 训练时，使用规模为 N 的模型训练至 L 所需的计算量临界值。这是

$$C_{\min}(C) \equiv \frac{C}{1 + B/B_{\text{crit}}(L)} \quad (\text{minimum compute, at } B \ll B_{\text{crit}}) \quad (5.5)$$

其中 $C = 6NBS$ 估计在 batch size B 下使用的（非嵌入）计算量。

5.2 $L(N, S_{\min})$ 的结果及性能随模型规模和计算量的变化

现在我们将使用方程 (5.4) 中定义的 $S_{\min}$ ，来获得在无限数据限制下损失对模型规模和训练时间依赖关系的简单普适拟合。我们将使用方程 (1.6) 来拟合稳定的、Adam 优化的训练运行，为方便起见在此重复：

$$L(N, S_{\min}) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{\min}}\right)^{\alpha_S} \quad (5.6)$$

⁵虽然临界 batch size 与梯度噪声尺度大致匹配，但我们在所有后续分析中使用的是从图 18 和 10 直接测量的 B_{crit} 。

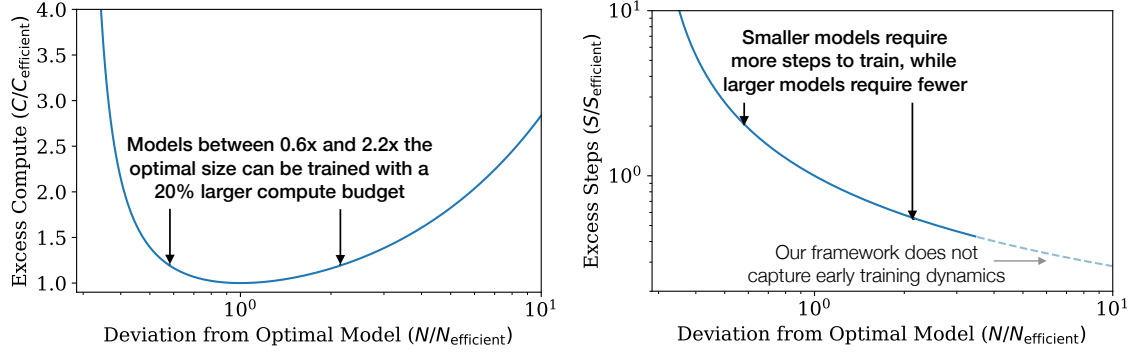


Figure 12 左: 给定固定的计算预算, 存在一个最优的模型规模, 虽然稍大或稍小的模型也可以以极少的额外计算量进行训练。右: 大于计算效率最优规模的模型需要更少的训练步数, 如果可以有足够的额外并行度, 则可能实现更快的训练。注意该方程不应被信赖用于非常大的模型, 因为它仅在初始过渡效应之后的学习曲线幂律区域内有效。

为损失的方程。我们包含学习率调度中 warmup 期之后的所有训练步数, 并使用以下参数拟合数据:

参数	α_N	α_S	N_c	S_c
值	0.077	0.76	6.5×10^{13}	2.1×10^3

Table 3 $L(N, S)$ 的拟合结果

使用这些参数, 我们在图 4 中获得了学习曲线拟合。虽然拟合并不完美, 但考虑到方程 (5.6) 的简洁性, 我们认为它们相当有说服力。

数据和拟合可以以不同且更有趣的方式可视化, 如图 11 所示。在该图中, 我们研究测试损失作为模型规模的函数, 同时固定训练所用的总非嵌入计算量 C 或步数 S 。对于拟合, 我们使用方程 (5.5) 和 (5.4) 以及上述参数和方程 (5.6)。

损失对 $S_{\min}$ 的幂律依赖反映了优化器动力学与损失景观之间的相互作用。由于拟合在训练后期最为准确, 此时损失可能近似为二次的, 因此该幂律应能提供关于损失 Hessian 谱的信息。其普适性表明 Hessian 特征值密度大致与模型规模无关。

5.3 Early Stopping 步数的下界

$L(N, S_{\min})$ 的结果可用于推导在数据受限的训练中应当进行 early stopping 的步数的下界 (以及粗略估计)。其动机源于如下思想: 对于给定模型, 有限和无限 D 的学习曲线在我们达到 $S_{\min} \approx S_{\text{stop}}$ 之前会非常相似。因此, 过拟合应与简单地在 S_{stop} 处停止训练的修正量成比例。这将低估 S_{stop} , 因为实际上当 D 有限时, 测试损失会下降得更慢, 因此我们需要更多训练步数才能在有限 D 下达到最优测试损失。这一推理线路导致了不等式:

$$S_{\text{stop}}(N, D) \gtrsim \frac{S_c}{[L(N, D) - L(N, \infty)]^{1/\alpha_S}} \quad (5.7)$$

其中 $L(N, \infty)$ 是在无限可用数据下评估的收敛损失。该不等式及其与经验数据的对比见附录图 16。在该图中, S_{stop} 和 $L(N, D)$ 的值是经验性的 (但 S_{stop} 已调整以模拟在 $B \gg B_{\text{crit}}$ 下的训练), 而 $L(N, \infty)$ 是从在 $D = \infty$ 处评估的 $L(N, D)$ 拟合计算得出的。

6 计算预算的最优分配

我们在图 1 的右上角展示了性能作为训练所用计算量函数的趋势。然而, 这一结果涉及在固定 batch size B 下训练, 而我们知道实际上可以通过以第 5.1 节讨论的 batch size B_{crit} 训练来实现更高效的训

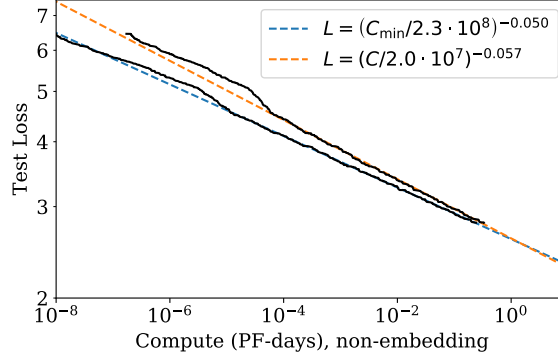


Figure 13 当调整性能以模拟远低于临界 batch size 的训练时，我们发现与纯经验结果相比， $L(C_{\min})$ 的幂律有所改变。 10^{-5} PF-days 处的明显凸起标志着从 1 层到 2 层网络的过渡；我们在幂律拟合中排除了 1 层网络。我们预期 $L(C_{\min})$ 趋势能够为更大的计算量提供可靠的外推。

练⁶。较大和较小的损失值本可以分别以更少的样本或更少的步数实现，通过标准化到临界 batch size 来纠正这一低效，可以得到更清晰、更可预测的趋势。

在本节中，我们将对此疏忽进行修正。更重要的是，我们将使用第 5 节的结果来确定计算量在模型规模 N 和训练过程中处理的数据量（即 $2B_{\text{crit}}S_{\min}$ ）之间的最优分配。我们将通过使用 $L(N, S_{\min})$ 方程，从经验和理论两方面确定这一分配，并证明这些方法是一致的。

6.1 最优性能与分配

我们首先研究损失作为来自方程 (5.5) 的最优分配计算量的函数。结果以及幂律拟合如图 13 所示。我们看到，与图 1 的计算量图相比，使用 $C_{\min}$ 的新拟合有所改善。

给定 $L(C_{\min})$ ，自然要问在给定训练计算量下能够提供最小损失的最优模型规模 $N(C_{\min})$ 是什么。最优模型规模如图 14 所示。我们观察到 $N(C_{\min})$ 可以很好地用幂律拟合

$$N(C_{\min}) \propto (C_{\min})^{0.73}. \quad (6.1)$$

在图 12 中，我们展示了训练次优规模模型的效果（见附录 B.4）。

根据定义 $C_{\min} \equiv 6NB_{\text{crit}}S$ ，因此我们可以使用 $N(C_{\min})$ 来提取更多结果。特别地，由于先前的拟合显示 $B \propto L^{-4.8}$ 且 $L \propto C_{\min}^{-0.05}$ ，我们可以得出结论 $B_{\text{crit}} \propto C_{\min}^{0.24}$ 。这使我们得出结论：最优步数将仅随计算量增长非常缓慢，即

$$S_{\min} \propto (C_{\min})^{0.03}, \quad (6.2)$$

与图 14 中的经验结果一致。实际上，测量到的指数足够小，以至于我们的结果甚至可能与指数为零一致。

因此我们得出结论：在通过最优计算分配来扩展语言建模时，我们应主要增加模型规模 N ，同时通过 $B \propto B_{\text{crit}}$ 扩大 batch size，而串行步数几乎无需增加。由于计算效率最优的训练使用相对较少的优化步数，因此有必要进一步研究加速早期训练动力学的方法。

6.2 从 $L(N, S_{\min})$ 得出的预测

$L(C_{\min})$ 和分配的结果可以从第 5 节得到的 $L(N, S_{\min})$ 方程中预测。给定我们的 $L(N, S_{\min})$ 方程，我们可以代入 $S_{\min} = \frac{C_{\min}}{6NB}$ ，然后在固定训练计算量的情况下找到损失作为 N 的函数的最小值。我们在附录 B 中详细执行了这一过程，并在其中提供了一些额外的预测。

对于损失作为训练计算量的函数，我们预测：

$$L(C_{\min}) = \left(\frac{C_{\min}}{C} \right)^{\alpha_C^{\min}} \quad (6.3)$$

⁶有人可能会问为什么不一开始就以 B_{crit} 训练。原因是它不仅依赖于模型，还依赖于我们希望达到的目标损失值，因此是一个移动的目标。

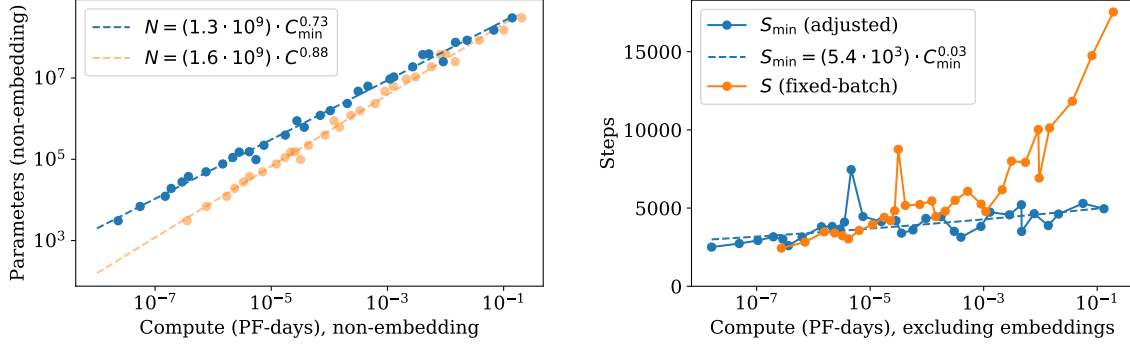


Figure 14 左: 每个计算预算 $C_{\min}$ 值都有一个关联的最优模型规模 N 。最优模型规模随 $C_{\min}$ 增长非常迅速, 计算量每增加 10 倍, 模型规模增长 5 倍。处理的数据样本数构成了增长的其余部分, 仅增长 2 倍, 相对温和。右: batch 调整后的优化步数也增长非常缓慢——甚至可能没有增长, 这意味着处理的数据样本增长中的大部分可用于增加 batch size。

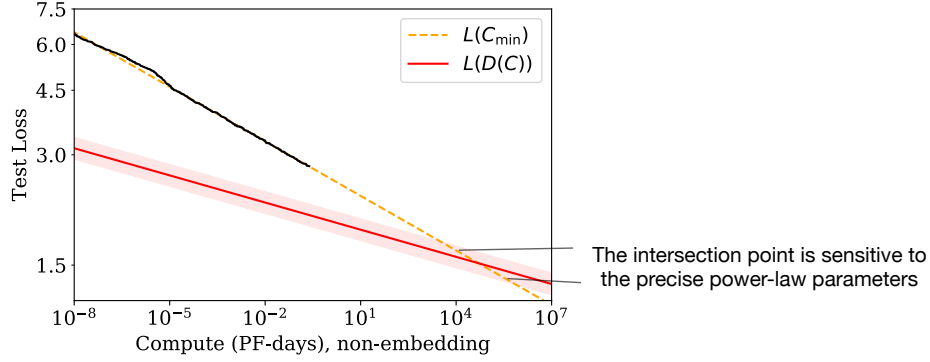


Figure 15 远超我们经验研究的模型规模后, 我们发现 $L(C_{\min})$ 和 $L(D)$ 的方程之间存在矛盾, 原因在于计算效率最优训练所需的数据增长缓慢。交点标志着我们预期预测在此点之前失效的位置。该点的位置对我们幂律拟合中的精确指数高度敏感。

其中

$$\alpha_C^{\min} \equiv \frac{1}{1/\alpha_S + 1/\alpha_B + 1/\alpha_N} \approx 0.054 \quad (6.4)$$

与图 13 的指数高度吻合。我们还预测

$$N(C_{\min}) \propto (C_{\min})^{\alpha_C^{\min}/\alpha_N} \approx (C_{\min})^{0.71} \quad (6.5)$$

这也与图 14 的缩放关系在几个百分点内一致。我们的 Scaling Laws 为语言建模的性能提供了一个预测框架。

6.3 矛盾与一个猜想

我们在计算量、数据或模型规模的较大值处没有观察到偏离直线幂律趋势的迹象。然而, 我们的趋势最终必须趋于平缓, 因为自然语言具有非零的熵。

实际上, 本节描述的计算效率最优训练趋势已经包含了一个明显的矛盾。在比本文记录高数个数量级的规模上, $L(C_{\min})$ Scaling Law 预测的性能将低于在训练数据随计算量缓慢增长的情况下应该可能的水平。这意味着我们的 Scaling Laws 必须在此点之前失效, 但我们推测交点具有更深层次的含义: 它提供了 Transformer 语言模型达到最大性能的点的估计。

由于计算效率最优训练所使用的数据量随计算预算增长缓慢, $L(C_{\min})$ 预测的性能最终会触碰到由 $L(D)$ 幂律设定的下界 (见图 15)。让我们更详细地推演这一点。

为了控制过拟合, 第 4 节的结果表明我们应按如下方式缩放数据集规模

$$D \propto N^{0.74} \propto C_{\min}^{0.54} \quad (6.6)$$

其中我们使用了图 14 中计算效率最优的 $N(C_{\min})$ 。

让我们将此与计算效率最优训练的数据需求进行比较。如果我们以临界 batch size 训练（即 $C = 2C_{\min}$ ）且在训练期间从不重复使用数据，我们发现数据使用量随计算量的增长为

$$D(C_{\min}) = \frac{2C_{\min}}{6N(C_{\min})} \approx (4 \times 10^{10} \text{ tokens}) (C_{\min}/\text{PF-Day})^{0.26} \quad (6.7)$$

这是数据集规模能够随计算量生产性增长的最大速率，因为这意味着我们仅训练一个 epoch。但它的增长速度比方程 (6.6) 中慢得多。这似乎意味着计算效率最优的训练最终将遇到过拟合的问题，即使训练过程从未重复使用任何数据也是如此！

根据图 1，我们预期当受到数据集规模瓶颈限制（即过拟合）时，损失应按 $L(D) \propto D^{-0.095}$ 缩放。这意味着一旦受数据限制，损失将按 $L(D(C_{\min})) \propto C_{\min}^{-0.03}$ 随计算量缩放。我们再次遇到矛盾，因为这将最终与我们在图 13 中对 $L(C_{\min})$ 的预测（缩放为 $L(C_{\min}) \propto C_{\min}^{-0.050}$ ）相交。

$L(D(C_{\min}))$ 和 $L(C_{\min})$ 的交点出现在

$$C^* \sim 10^4 \text{ PF-Days} \quad N^* \sim 10^{12} \text{ parameters}, \quad D^* \sim 10^{12} \text{ tokens}, \quad L^* \sim 1.7 \text{ nats/token} \quad (6.8)$$

尽管数值高度不确定，随着幂律拟合中精确指数值的不同，可能在每个方向上变化一个数量级。最明显的解释是，我们的 Scaling Laws 在达到这一点之前或达到时就会失效，而这一点在计算量和模型规模上仍有多数量级之远。

也可以推测这一交点具有更深层次的含义。如果我们不从根本上改变数据需求就无法将模型规模增加到 N^* 以上，也许这意味着一旦达到 $C_{\min}^*$ 和 N^* ，我们就已经提取了自然语言数据中所有可靠的信息。按照这种解读， L^* 将提供自然语言每 token 熵⁷的粗略估计。在这种情形下，我们将预期损失趋势在 L^* 处或之前趋于平缓。

我们可以通过考虑在训练数据集中加入噪声来猜测 $L(C_{\min})$ 趋于平缓时的函数形式。例如，我们可以在展示给模型的每个上下文后面附加一个随机 token 字符串，通过一个常数加性因子人为提高损失。那么，与噪声底线的距离 $L - L_{\text{noise}}$ 将是一个更有意义的性能指标，即使该距离的微小下降也可能代表定性性能的显著提升。由于人为噪声会平等地影响我们所有的趋势，6.8 的临界点不会改变（除了 L^* 的绝对值），即使它发生在平缓之后也可能是有意义的。

7 相关工作

幂律可以来源于各种各样的因素 [THK18]。密度估计 [Was06] 和随机森林模型 [Bia12] 中模型和数据集规模的幂律 Scaling 可能与我们的结果有关联。这些模型表明，幂律指数可以粗略地解释为数据中相关特征数量的倒数。

一些早期工作 [BB01, Goo01] 发现了性能与数据集规模之间的幂律缩放关系。更近期的工作 [HNA⁺17, HAD19] 也研究了模型规模与数据规模之间的缩放关系；他们的工作可能是文献中与我们最接近的⁸。然而，请注意 [HNA⁺17] 发现了数据集规模随模型规模的超线性缩放，而我们发现的是亚线性缩放。我们关于计算量最优分配的发现与 [Kom19] 之间存在一些平行之处，包括幂律学习曲线。EfficientNet [TL19] 似乎也遵循准确率与模型规模之间近似的幂律关系。最近的工作 [RRBS19b] 研究了多种数据集上随数据集规模和模型规模的缩放，并拟合了与我们类似的假设形式。

EfficientNet [TL19] 倡导为图像模型的最优性能以指数方式（具有不同系数）扩展深度和宽度，导致宽度作为深度函数的幂律缩放。我们发现对于语言模型，这个幂在扩展时应当大致为 1（因为宽度/深度应保持固定）。但更重要的是，我们发现精确的架构超参数与语言模型的总体规模相比是不重要的。在 [VWB16] 中，有人认为深层模型可以作为较浅模型的集成，这可能解释这一发现。更早期的工作 [ZK16] 比较了宽度和深度，发现宽 ResNets 在图像分类上可以超越深层 ResNets。一些研究固定每个数据样本的计算量，这倾向于与模型参数数量成比例，而我们则同时研究模型规模和训练计算量的 Scaling。

多篇工作 [AS17, BHMM18] 研究了高度过参数化模型中的泛化问题，发现当模型规模达到数据集规模时会出现堵塞相变 [GJS⁺19]（这可能需要在典型实践之外多训练许多数量级，特别是不使用 early stopping）。我们没有观察到这种相变，并发现必要的训练数据量按照模型规模的亚线性关系增长。模型规模的展开，特别是在大宽度下的展开 [JGH18, LXS⁺19]，可能为思考我们的一些 Scaling 关系提供

⁷使用 `wc` 工具定义单词，WebText2 数据集每个单词有 1.4 个 token，每个 token 有 4.3 个字符。

⁸本工作完成后，[RRBS19a] 也出现了，该文对损失对模型和数据集规模的依赖关系做出了类似的预测。

一个有用的框架。我们关于优化的结果，例如学习曲线的形状，可能可以用噪声二次模型来解释，该模型在实际设置中 [ZLN⁺19] 可以提供相当准确的预测。要在数量上建立这种联系，需要对 Hessian 谱 [Pap18, GKX19, GARD18] 进行刻画。

8 讨论

我们观察到语言模型对数似然损失与非嵌入参数数量 N 、数据集规模 D 和优化后的训练计算量 $C_{\min}$ 之间一致的缩放关系，如方程 (1.5) 和 (1.6) 所概括。另一方面，我们发现对许多架构和优化超参数的依赖非常弱。由于 $N, D, C_{\min}$ 的缩放是幂律的，随着规模的增加存在收益递减。

我们能够精确地建模损失对 N 和 D 以及损失对 N 和 S 的依赖关系（当这些参数同时变化时）。我们使用这些关系推导了计算 Scaling、过拟合的幅度、early stopping 步数以及训练大语言模型时的数据需求。因此，我们的 Scaling 关系不仅仅是观察结果，而是提供了一个预测框架。人们可以将这些关系理解为类似于理想气体定律，它以普适的方式关联气体的宏观性质，而与气体微观成分的大多数细节无关。

很自然地可以推测，这些 Scaling 关系将适用于其他具有最大似然损失的生成式建模任务，甚至可能适用于其他场景。为此，在其他领域（如图像、音频和视频模型，以及可能的随机网络蒸馏）上测试这些关系将是有趣的。目前我们不知道我们的哪些结果依赖于自然语言数据的结构，哪些是普适的。同样令人兴奋的是找到一个能够推导出这些 Scaling 关系的理论框架：一种支撑我们所观察到的热力学的统计力学。这样一种理论可能使得推导出其他更精确的预测成为可能，并提供对 Scaling Laws 局限性的系统性理解。

在自然语言领域，研究损失的持续改善是否能转化为相关语言任务上的改善将很重要。平滑的定量变化可以掩盖重大的质变：更多即不同。例如，经济的平滑总体增长不能表明支撑它的具体技术发展。类似地，语言模型损失的平滑改善可能隐藏着看似定性的能力变化。

我们的结果强烈表明，更大的模型将继续表现得更好，并且将比之前所理解的更具样本效率。大模型可能比大数据更重要。在此背景下，有必要对模型并行进行进一步的调研。深层模型可以使用流水线 [HCC⁺18] 进行训练，该方法在设备之间沿深度方向分割参数，但随着更多设备的使用，最终需要增加 batch size。另一方面，宽网络更易于并行化 [SCP⁺18]，因为大层可以在多个 worker 之间拆分，具有较少的串行依赖。稀疏性 [CGRS19, GRK17] 或分支架构（例如 [KSH12]）可能通过增强模型并行度实现大网络的更快训练。而使用像 [WRH17, WYL19] 这样的方法，即在训练过程中增长网络，可能使得整个训练过程都保持在计算效率前沿上。

致谢

我们感谢 Shan Carter、Paul Christiano、Jack Clark、Ajeya Cotra、Ethan Dyer、Jason Eisner、Danny Hernandez、Jacob Hilton、Brice Menard、Chris Olah 以及 Ilya Sutskever 的讨论和对本文草稿的反馈。

Appendices

A Summary of Power Laws

For easier reference, we provide a summary below of the key trends described throughout the paper.

Parameters	Data	Compute	Batch Size	Equation
N	∞	∞	Fixed	$L(N) = (N_c/N)^{\alpha_N}$
∞	D	Early Stop	Fixed	$L(D) = (D_c/D)^{\alpha_D}$
Optimal	∞	C	Fixed	$L(C) = (C_c/C)^{\alpha_C}$ (naive)
N_{opt}	D_{opt}	C_{min}	$B \ll B_{\text{crit}}$	$L(C_{\text{min}}) = (C_c^{\text{min}}/C_{\text{min}})^{\alpha_C^{\text{min}}}$
N	D	Early Stop	Fixed	$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$
N	∞	S steps	B	$L(N, S) = \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{S_c}{S_{\text{min}}(S, B)} \right)^{\alpha_S}$

Table 4

The empirical fitted values for these trends are:

Power Law	Scale (tokenization-dependent)
$\alpha_N = 0.076$	$N_c = 8.8 \times 10^{13}$ params (non-embed)
$\alpha_D = 0.095$	$D_c = 5.4 \times 10^{13}$ tokens
$\alpha_C = 0.057$	$C_c = 1.6 \times 10^7$ PF-days
$\alpha_C^{\text{min}} = 0.050$	$C_c^{\text{min}} = 3.1 \times 10^8$ PF-days
$\alpha_B = 0.21$	$B_* = 2.1 \times 10^8$ tokens
$\alpha_S = 0.76$	$S_c = 2.1 \times 10^3$ steps

Table 5

The optimal parameters for compute efficient training are given by:

Compute-Efficient Value	Power Law	Scale
$N_{\text{opt}} = N_e \cdot C_{\text{min}}^{p_N}$	$p_N = 0.73$	$N_e = 1.3 \cdot 10^9$ params
$B \ll B_{\text{crit}} = \frac{B_*}{L^{1/\alpha_B}} = B_e C_{\text{min}}^{p_B}$	$p_B = 0.24$	$B_e = 2.0 \cdot 10^6$ tokens
$S_{\text{min}} = S_e \cdot C_{\text{min}}^{p_S}$ (lower bound)	$p_S = 0.03$	$S_e = 5.4 \cdot 10^3$ steps
$D_{\text{opt}} = D_e \cdot C_{\text{min}}^{p_D}$ (1 epoch)	$p_D = 0.27$	$D_e = 2 \cdot 10^{10}$ tokens

Table 6

B Empirical Model of Compute-Efficient Frontier

Throughout this appendix all values of C , S , and α_C are adjusted for training at the critical batch size B_{crit} . We have left off the ‘adj’ label to avoid cluttering the notation.

B.1 Defining Equations

The power-law fit to the learning curves implies a simple prescription for compute-efficient training. In this appendix, we will derive the optimal performance, model size, and number of training steps as

a function of the compute budget. We start with the Equation (1.6), repeated here for convenience:

$$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S}\right)^{\alpha_S}. \quad (\text{B.1})$$

Here, S represents the number of parameter updates when training **at the critical batch size** [MKAT18], which was defined in Equation (5.2)⁹:

$$B(L) = \frac{B_*}{L^{1/\alpha_B}}. \quad (\text{B.2})$$

We would like to determine optimal training parameters for a fixed compute budget, so we replace $S = C / (6NB(L))$, where C is the number of FLOPs used in the training run:

$$L(N, C) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(6B_*S_c \frac{N}{L^{1/\alpha_B}C}\right)^{\alpha_S}. \quad (\text{B.3})$$

Now, we set $\partial_N L|_C = 0$ to find the condition for optimality:

$$\begin{aligned} 0 &= \frac{\partial L}{\partial N}|_C \\ &= -\frac{\alpha_N}{N} \left(\frac{N_c}{N}\right)^{\alpha_N} + \frac{\alpha_S}{N} \left(6B_*S_c \frac{N}{L^{1/\alpha_B}C}\right)^{\alpha_S} \left(1 - 5 \frac{N}{L} \frac{\partial L}{\partial N}|_C\right) \\ \Rightarrow \frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N}\right)^{\alpha_N} &= \left(6B_*S_c \frac{N}{L^{1/\alpha_B}C}\right)^{\alpha_S} \end{aligned} \quad (\text{B.4})$$

Equation (B.3) and (B.4) together determine the compute-efficient frontier.

B.2 Efficient Training

Now we assemble the implications of (B.3) and (B.4). First, note that inserting (B.4) into (B.3) yields

$$L(N_{\text{eff}}(C), C) = \left(1 + \frac{\alpha_N}{\alpha_S}\right) L(N_{\text{eff}}, \infty), \quad (\text{B.5})$$

which implies that for compute-efficient training, we should train to a **fixed percentage** $\frac{\alpha_N}{\alpha_S} \approx 10\%$ above the converged loss. Next, let's determine how the optimal loss depends on the compute budget. Eliminating N yields a power-law dependence of performance on compute:

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \quad (\text{B.6})$$

where we defined

$$\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052 \quad (\text{B.7})$$

$$C_c = 6N_c B_* S_c \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{1/\alpha_S + 1/\alpha_N} \left(\frac{\alpha_S}{\alpha_N}\right)^{1/\alpha_S}. \quad (\text{B.8})$$

Similarly, we can eliminate L to find $N(C)$:

$$\frac{N(C)}{N_c} = \left(\frac{C}{C_c}\right)^{\alpha_C/\alpha_N} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{1/\alpha_N} \quad (\text{B.9})$$

and

$$S(C) = \frac{C_c}{6N_c B_*} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{-1/\alpha_N} \left(\frac{C}{C_c}\right)^{\alpha_C/\alpha_S} \quad (\text{B.10})$$

⁹There is a slight ambiguity here: we can imagine training either at a constant batch size $B(L_{\text{target}})$, or we could instead train at a variable batch size $\tilde{B}(L)$, where $\tilde{B}$ is the instantaneous critical batch size (as opposed to B , which is the averaged version). These two prescriptions result in the same number of steps, so we can ignore this subtlety (see [MKAT18]).

B.3 Comparison to Inefficient

Typically, researchers train models until they appear to be close to convergence. In this section, we compare the efficient training procedure described above to this more typical setup. We define a the convergence factor f as the percent deviation from the converged loss:

$$L(N, C) = (1 + f) L(N, \infty). \quad (\text{B.11})$$

For compute-efficient training we have $f = \alpha_N / \alpha_S \approx 10\%$ from the previous section, but researchers typically use a much smaller value. Here, we choose $f' = 2\%$ as an estimate. For a fixed value of the loss, we predict:

$$\frac{N_f}{N_{f'}} = \left(\frac{1 + f}{1 + f'} \right)^{1/\alpha_N} \approx 2.7 \quad (\text{B.12})$$

$$\frac{S_f}{S_{f'}} = \left(\frac{1 + \frac{1}{f}}{1 + \frac{1}{f'}} \right)^{1/\alpha_S} \approx 0.13 \quad (\text{B.13})$$

$$\frac{C_f}{C_{f'}} = \frac{N_f}{N_{f'}} \frac{S_f}{S_{f'}} \approx 0.35 \quad (\text{B.14})$$

So that compute-efficient training uses 7.7x fewer parameter updates, 2.7x more parameters, and 65% less compute to reach the same loss.

B.4 Suboptimal Model Sizes

We can solve A.1 to find an expression for the amount of compute needed to reach a given value of the loss L with a model of size N :

$$C(N, L) = \left(6B_* S_c \frac{N}{L^{1/\alpha_B}} \right) \left(L - \left(\frac{N_c}{N} \right)^{\alpha_N} \right)^{-1/\alpha_S}. \quad (\text{B.15})$$

Using A.6 and A.9, we can eliminate L in favor of $N_{\text{eff}}(L)$, the model size which reaches L most efficiently. From there, we find an expression for the excess compute needed as a consequence of using a suboptimal model size:

$$\frac{C(N, N_{\text{eff}})}{C(N_{\text{eff}}, N_{\text{eff}})} = \frac{N}{N_{\text{eff}}} \left[1 + \frac{\alpha_S}{\alpha_N} \left(1 - \left(\frac{N_{\text{eff}}}{N} \right)^{\alpha_N} \right) \right]^{-1/\alpha_S}. \quad (\text{B.16})$$

The result is shown in Figure X. Models between 0.6x and 2.2x the optimal size can be used with only a 20% increase in compute budget. Using a smaller model is useful when accounting for the cost inference. A larger model can be trained the the same level of performance in fewer steps, allowing for more parallelism and faster training if sufficient hardware is available (see Figure Y):

$$\frac{S(N, N_{\text{eff}})}{S(N_{\text{eff}}, N_{\text{eff}})} = \left[1 + \frac{\alpha_S}{\alpha_N} \left(1 - \left(\frac{N_{\text{eff}}}{N} \right)^{\alpha_N} \right) \right]^{-1/\alpha_S}. \quad (\text{B.17})$$

A 2.2x larger model requires 45% fewer steps at a cost of 20% more training compute. Note that this equation should not be trusted for very large models, as it is only valid in the power-law region of the learning curve after initial transient effects.

C Caveats

In this section we list some potential caveats to our analysis.

- At present we do not have a solid theoretical understanding for any of our proposed scaling laws. The scaling relations with model size and compute are especially mysterious. It may be possible to understand scaling at very large D holding model size fixed [AS17], and also the shape of learning curves late in training, by modeling the loss with a noisy quadratic. But the scaling with D at very large model size still remains mysterious. Without a theory or a systematic understanding of the corrections to our scaling laws, it's difficult to determine in what circumstances they can be trusted.

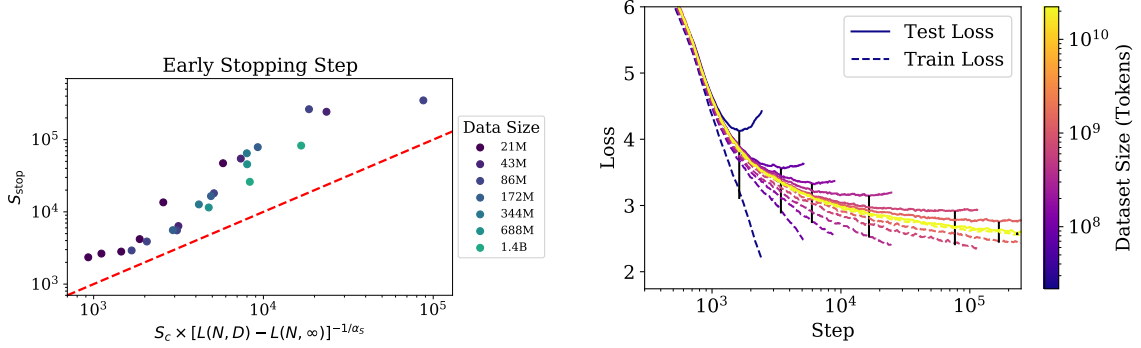


Figure 16 **Left:** We characterize the step on which early stopping occurs, as a function of the extent of overfitting. The red line indicates a lower bound for early stopping that is derived in Section 5.3. **Right:** We display train and test loss for a series of 300M parameter models trained on different sized dataset sub-samples. The test loss typically follows that of a run done with unrestricted data until diverging. Note that the degree of overfitting (as compared to the infinite data limit) is significantly overestimated by $L_{\text{test}} - L_{\text{train}}$ (denoted by a black bar for each run).

- We are not especially confident in the prediction of $B_{\text{crit}}(L)$ for values of the loss far outside the range we have explored. Changes in B_{crit} could have a significant impact on trade-offs between data parallelism and the number of serial training steps required, which would have a major impact on training time.
- We did not thoroughly investigate the small data regime, and our fits for $L(N, D)$ were poor for the smallest values of D (where an epoch corresponded to only 40 steps). Furthermore, we did not experiment with regularization and data augmentation. Improvements in these could alter our results, quantitatively or qualitatively.
- We used the estimated training compute $C \approx 6NBS$, which did not include contributions proportional to n_{ctx} (see Section 2.1). So our scalings with compute may be confounded in practice in the regime of very large n_{ctx} , specifically where $n_{\text{ctx}} \gtrsim 12d_{\text{model}}$.
- We tuned learning rates, and we experimented with learning rate schedules. But we may have neglected to tune some hyperparameter (e.g. initialization scale or momentum) that have an important effect on scaling.
- The optimal choice of learning rate is sensitive to the target loss. When training close to convergence, it may be necessary to use a smaller learning rate to avoid divergences. But when conducting a short training run (eg due to compute limitations), it may be possible to use a larger learning rate. We did not experiment with higher learning rates for training runs that did not proceed to convergence.

D Supplemental Figures

D.1 Early Stopping and Test vs Train

In section 5.3 we described the result shown in Figure 16, which provides a prediction for a lower bound on the early stopping step. We also show the train and test loss for a given model size when training on different sized datasets.

D.2 Universal Transformers

We compare the performance of standard Transformers to recurrent Transformers [DGV⁺18] in Figure 17. These models re-use parameters, and so perform slightly better as a function of N , but slightly worse as a function of compute C . We include several different possibilities for parameter re-use.

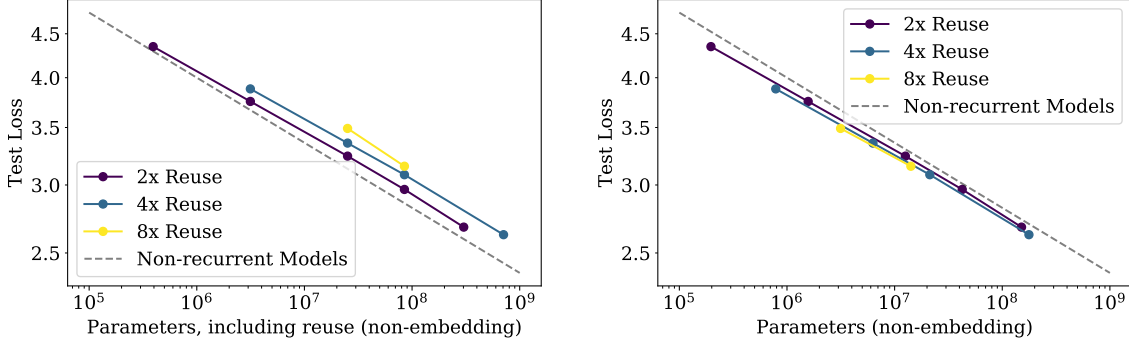


Figure 17 We compare recurrent Transformers [DGV⁺18], which re-use parameters, to standard Transformers. Recurrent Transformers perform slightly better when comparing models with equal parameter count, but slightly worse when accounting for reuse and comparing per FLOP.

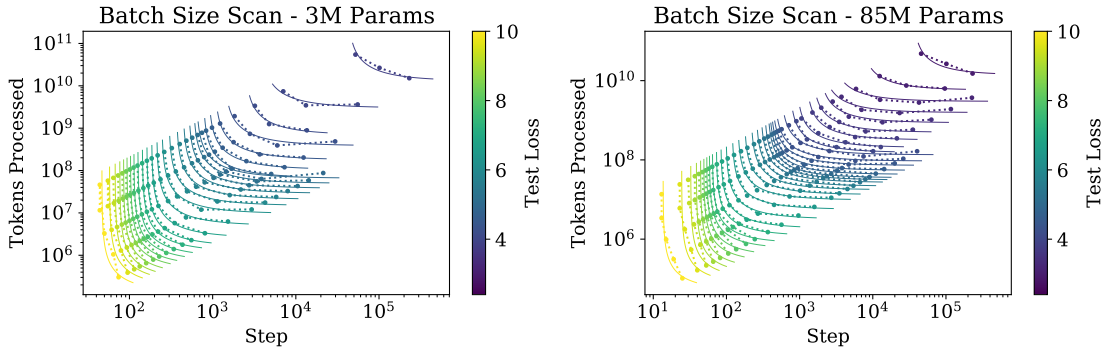


Figure 18 These figures demonstrate fits to Equation (5.1) for a large number of values of the loss L , and for two different Transformer model sizes. These fits were used to measure $B_{\text{crit}}(L)$ for Figure 10.

D.3 Batch Size

We measure the critical batch size using the data displayed in figure 18. This made it possible to estimate $B_{\text{crit}}(L)$ in figure 10.

D.4 Sample Efficiency vs Model Size

It is easy to see from figure 2 that larger models train faster, and are therefore more sample efficient. We provide another way of looking at this phenomenon in figure 19, which shows when different models reach various fixed values of the loss.

D.5 Context Dependence

The trends for loss as a function of model size are displayed for different tokens in the context in Figure 21. We see that models trained on $n_{\text{ctx}} = 1024$ show steady improvement with model size on all but the first token.

Fixing model size, it appears that the loss scales as a power-law as a function of position T in the context, see Figure 20. This may be a consequence of underlying power-law correlations in language [EP94, ACDE12, LT16], or a more general feature of the model architecture and optimization. It provides some suggestion for the potential benefits (or lack thereof) from training on larger contexts. Not only do larger models converge to better performance at $T = 1024$, but they also improve more quickly at early tokens, suggesting that larger models are more efficient at detecting patterns with less contextual information. In the right-hand plot we show how per-token performance varies for a

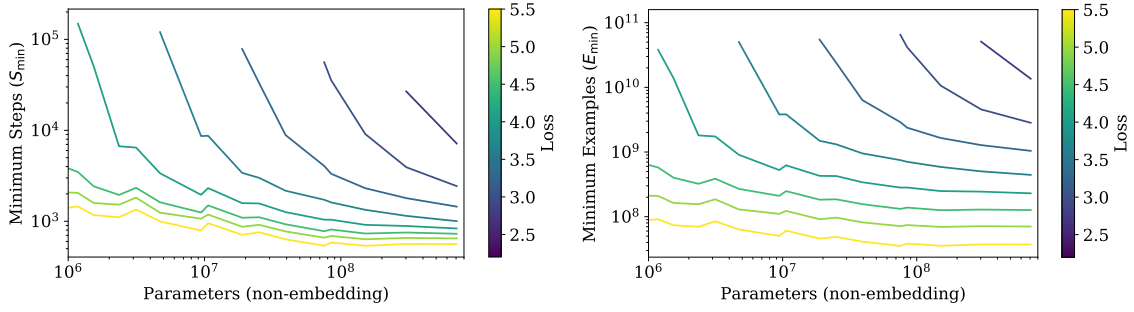


Figure 19 The number of minimum serial steps needed to reach any fixed value of the test loss decreases precipitously with model size. Sample efficiency (show here for training far below the critical batch size) improves greatly as well, improving by a factor of almost 100 when comparing the smallest possible model to a very large one.

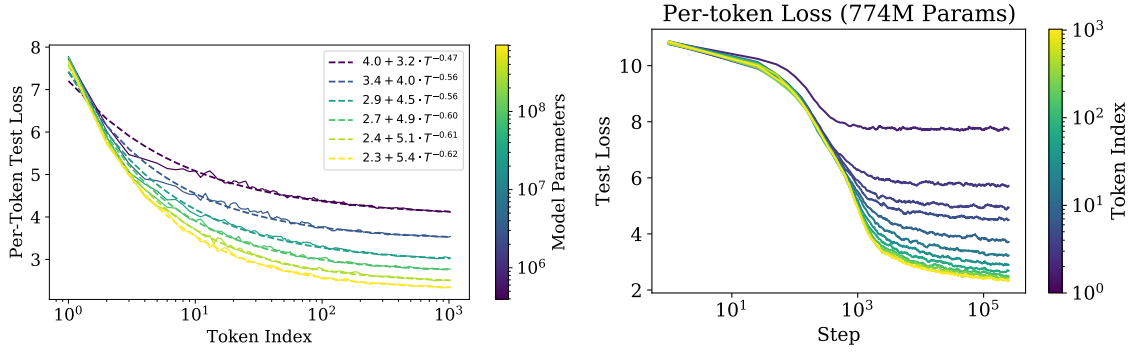


Figure 20 This figure provides information about the performance per token as a function of model size and training time. **Left:** Loss per token as a function of its position T in the 1024-token context. Loss scales predictably as a power-law in T . **Right:** Test loss per token as a function of training step.

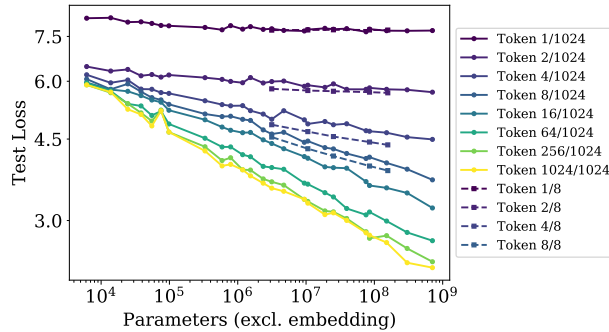


Figure 21 In addition to the averaged loss, individual tokens within the 1024-token context also improve smoothly as model size increases. Training runs with shorter context $n_{\text{ctx}} = 8$ (dashed lines) perform better on early tokens, since they can allocate all of their capacity to them.

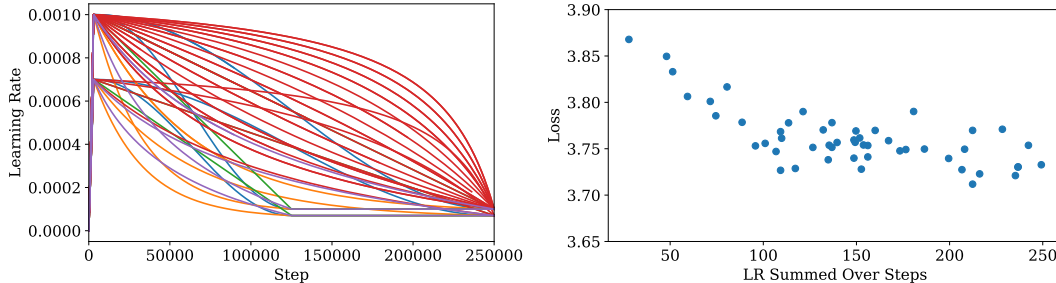


Figure 22 We test a variety of learning rate schedules including cosine decay, linear decay, as well as other faster/slower decays schedules on a 3 million parameter model, shown on the left. For these experiments we do not decay to zero, since we find that this tends to give a fixed improvement close to the end of training. We find that, as long as the learning rate is not too small and does not decay too quickly, performance does not depend strongly on learning rate. Run-to-run variation is at the level of 0.05 in the loss, so averaging multiple runs is necessary to validate performance changes smaller than this level.

fixed model as a function of the training step. The model begins by learning short-range information, and only learns longer-range correlations later in training.

We have also included models trained with a tiny context $n_{\text{ctx}} = 8$ in order to compare with our longer context models. Even modestly sized models trained on $n_{\text{ctx}} = 8$ can dominate our largest $n_{\text{ctx}} = 1024$ models on very early tokens. This also suggests that further improvements should be possible with much larger models trained on large contexts.

D.6 Learning Rate Schedules and Error Analysis

We experimented with a variety of learning rates and schedules. A host of schedules and resulting test performances for a small language model are plotted in Figure 22. We conclude that the choice of learning rate schedule is mostly irrelevant, as long as the total summed learning rate is sufficiently large, and the schedule includes a warmup period and a final decay to near-vanishing learning rate. Variations among schedules appear to be statistical noise, and provide a rough gauge for the scale of variation between different training runs. Experiments on larger models suggest that the variation in the final test loss between different random seeds is roughly constant in magnitude for different model sizes.

We found that larger models require a smaller learning rate to prevent divergence, while smaller models can tolerate a larger learning rate. To implement this, the following rule of thumb was used for most runs:

$$\text{LR}(N) \approx 0.003239 + -0.0001395 \log(N) \quad (\text{D.1})$$

We expect that this formula could be improved. There may be a dependence on network width, likely set by the initialization scale. The formula also breaks down for $N > 10^{10}$ parameters. Nevertheless, we found that it works sufficiently well for the models we considered.

D.7 Fit Details and Power Law Quality

We experimented with a number of functional forms for the fits to $L(N)$, $L(C)$, and $L(D)$; the power-law fits were qualitatively much more accurate than other functions such as logarithms (see Figure 23).

For $L(C)$, we do not include small models with only 1 layer in the fit, as the transition from 1 to 2 layers causes a noticeable lump in the data. For $L(N)$ we also do not include very small models with only 1 layer in the fit, and we exclude the largest models that have not trained fully to convergence. Fit parameters change marginally if we do include them, and the trend extrapolates well in both directions regardless.

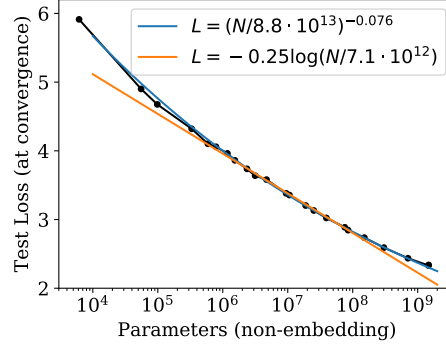


Figure 23 The trend for performance as a function of parameter count, $L(N)$, is fit better by a power law than by other functions such as a logarithm at a qualitative level.

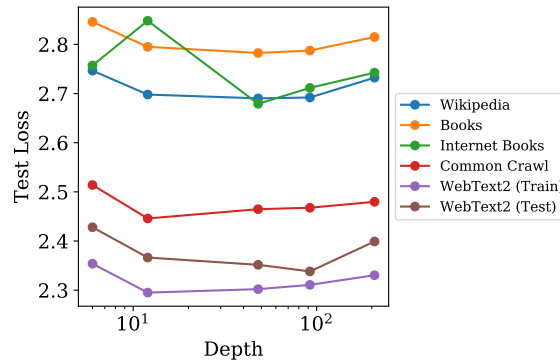


Figure 24 We show evaluations on a series of datasets for models with approximately 1.5 Billion parameters. We observe no effect of depth on generalization; generalization performance depends primarily on training distribution performance. The 12-layer model overfit the Internet Books dataset and we show the early-stopped performance; we have not seen this surprising result in other experiments.

D.8 Generalization and Architecture

In figure 24 we show that generalization to other data distributions does not depend on network depth when we hold the total parameter count fixed. It seems to depend only on the performance on the training distribution.

List of Figures

1	Summary of simple power laws.	3
2	样本效率与计算效率的说明	3
3	如何扩展模型规模、batch size 和串行步数	4
4	同时改变模型和数据规模, 或同时改变模型和训练步数时的性能	5
5	性能对超参数调优的弱依赖关系	7
6	包含或排除嵌入参数时性能趋势的比较	8
7	LSTM 与 Transformer 性能比较	8
8	对其他测试数据集的泛化	9
9	过拟合的普适性	10
10	Critical batch size	11

11	性能与计算预算或参数更新次数的关系	12
12	在次优模型上的训练	13
13	经验计算趋势与调整后计算趋势的比较	14
14	最优模型规模和串行步数与计算预算的关系	15
15	计算趋势与数据趋势之间的矛盾	15
16	Early stopping lower bound and training curves for overfit models	21
17	Universal transformers	22
18	Batch size scans	22
19	Another look at sample efficiency	23
20	Power-law dependence of performance on position in context	23
21	Performance at different context positions versus model size	23
22	Learning rate schedule scan	24
23	Comparison of Power-Law and Logarithmic Fits	25
24	Generalization versus depth	25

List of Tables

1	Transformer 的参数与计算量统计	6
2	$L(N, D)$ 的拟合结果	10
3	$L(N, S)$ 的拟合结果	13
4	Key trend equations	18
5	Key parameters to trend fits	18
6	Trends for compute-efficient training	18

References

- [ACDE12] Eduardo G Altmann, Giampaolo Cristadoro, and Mirko Degli Esposti. On the origin of long-range correlations in texts. *Proceedings of the National Academy of Sciences*, 109(29):11582–11587, 2012. 22
- [AS17] Madhu S. Advani and Andrew M. Saxe. High-dimensional dynamics of generalization error in neural networks. *arXiv*, 2017, 1710.03667. 10, 16, 20
- [BB01] Michele Banko and Eric Brill. Scaling to very very large corpora for natural language disambiguation. In *Proceedings of the 39th annual meeting on association for computational linguistics*, pages 26–33. Association for Computational Linguistics, 2001. 16
- [BHMM18] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine learning and the bias-variance trade-off. *arXiv*, 2018, 1812.11118. 16
- [Bia12] GÅŠrard Biau. Analysis of a random forests model. *Journal of Machine Learning Research*, 13(Apr):1063–1095, 2012. 16
- [CGRS19] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019, 1904.10509. URL <http://arxiv.org/abs/1904.10509>. 17
- [DCLT18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv:1810.04805. 2
- [DGV⁺18] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. *CoRR*, abs/1807.03819, 2018, 1807.03819. URL <http://arxiv.org/abs/1807.03819>. 5, 6, 8, 21, 22

- [EP94] Werner Ebeling and Thorsten Pöschel. Entropy and long-range correlations in literary english. *EPL (Europhysics Letters)*, 26(4):241, 1994. 22
- [Fou] The Common Crawl Foundation. Common crawl. URL <http://commoncrawl.org>. 7
- [GARD18] Guy Gur-Ari, Daniel A. Roberts, and Ethan Dyer. Gradient descent happens in a tiny subspace. 2018, arXiv:1812.04754. 17
- [GJS⁺19] Mario Geiger, Arthur Jacot, Stefano Spigler, Franck Gabriel, Levent Sagun, Stéphane d’Ascoli, Giulio Biroli, Clément Hongler, and Matthieu Wyart. Scaling description of generalization with number of parameters in deep learning. *arXiv*, 2019, 1901.01608. 16
- [GKX19] Behrooz Ghorbani, Shankar Krishnan, and Ying Xiao. An investigation into neural net optimization via hessian eigenvalue density. *CoRR*, abs/1901.10159, 2019, 1901.10159. URL <http://arxiv.org/abs/1901.10159>. 17
- [Goo01] Joshua Goodman. A bit of progress in language modeling. *CoRR*, cs.CL/0108005, 2001. URL <http://arxiv.org/abs/cs.CL/0108005>. 16
- [GRK17] Scott Gray, Alec Radford, and Diederik P Kingma. Gpu kernels for block-sparse weights. *openai.com*, 2017. 17
- [HAD19] Joel Hestness, Newsha Ardalani, and Gregory Diamos. Beyond human-level accuracy: Computational challenges in deep learning. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, PPOPP ’19, pages 1–14, New York, NY, USA, 2019. ACM. doi:10.1145/3293883.3295710. 16
- [HCC⁺18] Yanping Huang, Yonglong Cheng, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *CoRR*, abs/1811.06965, 2018, 1811.06965. URL <http://arxiv.org/abs/1811.06965>. 17
- [HNA⁺17] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md. Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically, 2017, 1712.00409. 16
- [JGH18] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *Advances in neural information processing systems*, pages 8571–8580, 2018. 16
- [KB14] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014, 1412.6980. 6
- [Kom19] Aran Komatsuzaki. One epoch is all you need, 2019, arXiv:1906.06669. 16
- [KSH12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1*, NIPS’12, pages 1097–1105, USA, 2012. Curran Associates Inc. URL <http://dl.acm.org/citation.cfm?id=2999134.2999257>. 17
- [LCG⁺19] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2019, 1909.11942. 8
- [LOG⁺19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019, 1907.11692. URL <http://arxiv.org/abs/1907.11692>. 2
- [LSP⁺18] Peter J. Liu, Mohammad Saleh, Etienne Pot, Ben Goodrich, Ryan Sepassi, Lukasz Kaiser, and Noam Shazeer. Generating wikipedia by summarizing long sequences. *arXiv:1801.10198 [cs]*, 2018, 1801.10198. URL <http://arxiv.org/abs/1801.10198>. 2, 6
- [LT16] Henry W Lin and Max Tegmark. Criticality in formal languages and statistical physics. *arXiv preprint arXiv:1606.06737*, 2016. 22
- [LXS⁺19] Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent, 2019, arXiv:1902.06720. 16

- [MKAT18] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training, 2018, arXiv:1812.06162. 4, 5, 11, 12, 19
- [Pap18] Vardan Papyan. The full spectrum of deep net Hessians at scale: Dynamics with sample size. *CoRR*, abs/1811.07062, 2018, 1811.07062. URL <http://arxiv.org/abs/1811.07062>. 17
- [RNSS18] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. URL https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/languageunsupervised/language_understanding_paper.pdf, 2018. 2, 6
- [RRBS19a] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales, 2019, 1909.12673. 16
- [RRBS19b] Jonathan S. Rosenfeld, Amir Rosenfeld, Yonatan Belinkov, and Nir Shavit. A constructive prediction of the generalization error across scales, 2019, arXiv:1909.12673. 16
- [RSR⁺19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2019, arXiv:1910.10683. 2
- [RWC⁺19] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *openai.com*, 2019. 2, 4, 6, 7
- [SCP⁺18] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, Hyukjoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake Hechtman. Mesh-tensorflow: Deep learning for supercomputers, 2018, 1811.02084. 17
- [SHB15] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *CoRR*, 2015, 1508.07909. 6
- [SLA⁺18] Christopher J. Shallue, Jaehoon Lee, Joe Antognini, Jascha Sohl-Dickstein, Roy Frostig, and George E. Dahl. Measuring the effects of data parallelism on neural network training, 2018, arXiv:1811.03600. 11
- [SS18] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. *CoRR*, abs/1804.04235, 2018, 1804.04235. URL <http://arxiv.org/abs/1804.04235>. 6
- [THK18] Stefan Thurner, Rudolf Hanel, and Peter Klimek. *Introduction to the theory of complex systems*. Oxford University Press, 2018. 16
- [TL19] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946, 2019, 1905.11946. URL <http://arxiv.org/abs/1905.11946>. 16
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 5998–6008. Curran Associates, Inc., 2017. URL <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>. 2, 6
- [VWB16] Andreas Veit, Michael Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks, 2016, arXiv:1605.06431. 7, 16
- [Was06] Larry Wasserman. *All of nonparametric statistics*. Springer Science & Business Media, 2006. 16
- [WPN⁺19] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2019, 1905.00537. 2
- [WRH17] Yu-Xiong Wang, Deva Ramanan, and Martial Hebert. Growing a brain: Fine-tuning by increasing model capacity. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul 2017. doi:10.1109/cvpr.2017.323. 17

- [WYL19] Wei Wen, Feng Yan, and Hai Li. Autogrow: Automatic layer growing in deep convolutional networks, 2019, 1906.02909. 17
- [YDY⁺19] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. Xlnet: Generalized autoregressive pretraining for language understanding, 2019, arXiv:1906.08237. 2
- [ZK16] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *Proceedings of the British Machine Vision Conference 2016*, 2016. doi:10.5244/c.30.87. 16
- [ZKZ⁺15] Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. *2015 IEEE International Conference on Computer Vision (ICCV)*, Dec 2015. doi:10.1109/iccv.2015.11. 7
- [ZLN⁺19] Guodong Zhang, Lala Li, Zachary Nado, James Martens, Sushant Sachdeva, George E. Dahl, Christopher J. Shallue, and Roger B. Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. *CoRR*, abs/1907.04164, 2019, 1907.04164. URL <http://arxiv.org/abs/1907.04164>. 11, 17